



**UNIVERSITÀ
DEGLI STUDI
DI TRIESTE**



Universitat d'Alacant

**Department of Mathematics, Informatics and Geosciences
MASTER DEGREE IN DATA SCIENCE AND ARTIFICIAL INTELLIGENCE**

MASTER THESIS

Underwater Search and Navigation in Realistic Environment

Candidate:

Christian Faccio

Supervisor:

Antonio Celani

Co-supervisor:

Giulia De Masi

ACADEMIC YEAR 2025 – 2026

Underwater Search and Navigation in Realistic Environment

by

Christian Faccio

Submitted to the Department of Mathematics, Informatics and Geosciences
in partial fulfillment of the requirements for the degree of

MASTER DEGREE IN DATA SCIENCE AND ARTIFICIAL INTELLIGENCE

ABSTRACT

Coral reefs are an existential habitat for a wide range of marine species, offering shelter, food and breeding grounds. However, they are highly sensitive to environmental changes, and are increasingly threatened by climate change and human activities. When preservation is not possible, restoration is seen as a promising alternative to regenerate the biome and preserve the marine biodiversity. Current approaches and techniques for coral restoration can't avoid an intrinsic risk for human divers. For these reasons, the use of autonomous underwater vehicles (AUVs) has been proposed as a solution to overcome the limitations of human divers and to increase safety and efficiency in the search task. Moreover, given the unknown and complex nature of the marine environment, MARL algorithms have been increasingly used in these kind of tasks. This work aims to test and evaluate the use of a suitable MARL algorithm in a physically realistic simulated environment, where the agents are required to navigate and search for suitable target zones defined by a set of environmental characteristics. The results show that the proposed MARL algorithm is able to learn an optimal policy to navigate an unknown environment and find the target zones, outperforming a random policy. Moreover, the use of independent agents lead to better performance overall, against the use of a centralized critic or communication at execution time, even though in specific scenarios their use can be beneficial and lead to higher efficiency.

Contents

<i>List of Figures</i>	9
<i>List of Tables</i>	11
1 Introduction	13
2 Theoretical Background	17
2.1 Reinforcement Learning	17
2.1.1 Markov Decision Process	17
2.1.2 Value function	18
2.1.3 PPO	18
2.2 Multi-agent Reinforcement Learning	21
2.2.1 Markov Games	21
2.2.2 Cooperative setting	23
2.2.3 Challenges of MARL	23
3 Problem formulation	25
3.1 SwarmSwIM Simulator	25
3.2 Agent	26
3.3 Environment	27
3.3.1 Currents	28
3.3.2 Salinity Field	31
4 Proposed Method	33
4.1 IPPO	34
4.2 MAPPO	34
4.3 Agent State	35
4.4 Reward Function	37
4.5 Results	38
5 Experiments	43
5.1 Communication and CTDE	43
5.2 Agent initialization	45
5.3 Difficult targets	46
5.4 Communication range	47
5.5 Scaling N	47
5.6 Time to success	48

6	Conclusions and Future Work	51
	<i>References</i>	53

List of Figures

2.1	MDP framework.	18
3.1	Representation of the 6 DoF of an AUV [24].	26
3.2	The 27 discrete actions, decomposed by the vertical component Δz into three horizontal keypads. Each cell shows the commanded direction and its action index $a = 9(\Delta x + 1) + 3(\Delta y + 1) + (\Delta z + 1)$, where Δx is surge and Δy is sway in the body frame. Because z is measured positive downward, $\Delta z = +1$ descends. The centre cell of each keypad is one of the three purely vertical actions, with $a = 13$ the no-op. Normalising every triple to unit length makes the commanded speed v identical in all 26 directions, so diagonal moves cover no more ground per step than axis-aligned ones.	27
3.3	Currents field at four different depths, with the color representing the magnitude of the velocity. The arrows represent the direction of the current. . . .	30
3.4	Currents statistics. The first plot shows the aggregate horizontal velocity magnitude distribution at varying depth. The second plot shows the vertical current field magnitude at fixed y , varying x and depth. The third plot shows the overall horizontal magnitude distribution. The dark line refers to the agent's speed.	30
3.5	Salinity field of a simulation. The left plot shows the salinity field in the 3D environment, with the color representing the PSU value. The right plot adds also the target zone, defined as the points that satisfy a salinity value of 43.852 ± 0.15 and a turbidity value of 0.4194 ± 0.05	31
4.1	Training curves for 2 agent policies.	39
4.2	Training curves for 4 agent policies.	39
4.3	Trajectories of two agents initialized with <i>max_dist</i> and using an IPPO policy without communication.	40
4.4	Trajectories of 4 agents initialized with <i>max_dist</i> and using a MAPPO policy without communication.	41
4.5	Success rate for the different test scenarios.	41
5.1	Results of using communication (left column) and CTDE (right column). The first row refers to scenarios with targets sampled randomly, while the second row refers to scenarios with targets sampled from the tail of the salinity distribution at sampled depth.	44

5.2	Effect of different initialization methods on success rate (first row) and steps to success (second row). The first column refers to 2 agent tests, while the right one to 4 agent tests. The shadowed lines are the marginalization of the bold ones on the two target methods, where the upper ones refer to random targets.	45
5.3	Results of using randomly sampled targets (left column) and targets sampled from the tail of the salinity distribution at sampled depth (right column). The first row refers to 2 agents' cases, while the second one to 4 agents' cases. The plots show the fraction of solved episodes as a function of the step budget. .	46
5.4	Effect of different communication ranges to the success rate on 2 agents (left plot) and 4 agents (right plot). The shadowed lines are the marginalization of the bold ones on the two target methods, where the upper ones refer to random targets.	47
5.5	Average success rate using 2 agents (left plot) and 4 agents (right plot). The grey line refers to the random agent.	48
5.6	Median times to succeed for the first agent only (left column) and the relative normalized metric (right column), which is computed as $1 - \mathbb{E}(\min(t, T))/T$, where T refers to the <i>max_steps</i> parameter. The first row refers to 2 agents' cases, while the second row to 4 agents' ones.	49
5.7	Probabilities to arrive at target by function of the number of steps for the first (left column) and second (right column) agent for the policies trained with 2 agents. The first row refers to cases where agents are initialized uniformly at land, the second one to cases where agents are initialized at the environment origin (0, 0).	50
5.8	Probabilities to arrive at target by function of the number of steps for the first (first column), second (second column), third (third column) and fourth (fourth column) agent for the policies trained with 4 agents. The first row refers to cases where agents are initialized uniformly at land, the second one to cases where agents are initialized at the environment origin (0, 0).	50

List of Tables

3.1	Parameters used in the Oceananigans nonhydrostatic model	28
3.2	Overall currents' statistics in the training simulations	31
3.3	Overall salinity statistics in the training simulations	32
4.1	Default env parameters	36
4.2	IPPO parameters	37

Chapter 1

Introduction

Coastal regions are home to a significant portion of the global population, and they are increasingly affected by human activities. Desalination processes [1] have become the main solution to provide freshwater to many of these regions, and even though more efficient techniques have been developed, the accompanying environmental issues have been overlooked, in particular the consequences of the discharge of desalination by-products, such as the effects of concentrate effluent on the marine coastal ecosystem. Moreover, other industrial activities, such as offshore oil and gas operations [2], have been shown to have a significant impact on the marine environment, including the release of pollutants and the disruption of marine habitats. In fact, with the depletion of easily accessible oil and gas resources, offshore drilling is moving into deeper waters and more sensitive areas, without being matched by legislation that reflects modern practices of environmental conservation. Such regulations may help to increase environmental monitoring and protect sensitive marine areas and species. Therefore, the preservation and monitoring of marine environments is a strategic priority for coastal regions, especially in the Arabian Gulf, where the combination of high salinity, warm temperatures, and human activities poses a significant threat to marine ecosystems [3].

The Gulf's marine communities are additionally exposed to decades of accelerating coastal urbanization, identified as the single largest anthropogenic stressor acting on the region, ahead of oil and gas extraction and desalination combined [4], with land reclamation and dredging generating persistent turbidity plumes and direct habitat loss along the coastlines of Bahrain [5] and the wider Arabian Gulf [6]. Desalination brine discharge, while its impact on basin-scale salinity remains modest and concentrated near coastal outfalls rather than spread evenly across the basin [7], combines with recurring thermal stress to drive bleaching events on the last remaining reefs of the region [8]. These compounding, spatially localized pressures are precisely why salinity and turbidity, the two variables this thesis tasks the swarm with locating, are useful proxies for anthropogenic disturbance rather than arbitrary target quantities.

Coral reefs are one of the most biodiverse ecosystems on Earth, providing an existential habitat for a wide range of marine species, and offering shelter, food and breeding ground. Worldwide, billions of people depend more or less directly on coral reefs for a significant portion of their livelihoods, including fishing, tourism and coastal protection. Even if not directly sustaining human life in many wealthier regions of the globe, coral reefs provide an enormous economic value. Clearly, they are essential for the health of the marine environment

and the well-being of human societies, and should then be preserved and protected [9]. Unfortunately, they are highly sensitive to environmental changes, and their degradation could have severe consequences for the environment and the economy. Coral preservation is the best way to protect them, but is not always a viable option. Therefore, coral restoration is advocated as a promising tool to reduce coral loss and restore damaged reefs, but active interventions are needed to avoid the collapse of coral reefs worldwide in the next decades [10]. Many restoration techniques have been developed and many studies have been conducted to evaluate their effectiveness, but the success of these methods is still limited [11]. 30 – 40% of projects fail due to poor planning, unrealistic objectives, inadequate regular maintenance and persistent anthropogenic pressures. Moreover, restoration sites are often clustered around human settlements, which are not always the most suitable locations for coral growth, but limits human hazards and encourages safety for divers, while also reducing costs. While the ultimate goal of coral restoration is to return ecosystems to pre-disturbance states, achieving this goal is becoming increasingly difficult due to the complex and dynamic nature of marine environments, as well as the increasingly limited availability of suitable restoration sites. Finally, the search for suitable restoration sites and the further monitoring of the marine environment health is a time-consuming and dangerous task for human divers, who are exposed to different hazards and provide limited spatial coverage.

For these reasons, the use of autonomous underwater vehicles (AUVs) has been proposed as a solution to overcome the limitations of human divers and to increase safety and efficiency in the search task. In fact, with the recent technological advancements, swarms of underwater robots could provide seamless and continuous monitoring and spatial search capabilities, while also reducing costs. They are able to collect data from the environment, like temperature, salinity and turbidity, which are critical to assess habitat suitability for coral restoration. However, the use of AUVs in marine environments is still a challenging task, due to the complex and dynamic nature of the environment, as well as the limitations of current technologies. In fact, AUVs struggle in communicating with each other, due to the absence of advanced technologies for underwater communication. BlueME [12] is a compact magnetoelectric antenna array system for underwater robot-to-robot communication, which is able to provide a reliable signal transmission at distances beyond 700 meters while consuming less than 10 W of power. It demonstrates that communication in the kilometer range is possible, but the search area usually exceeds this range. Moreover, the robot usually can not rely on GPS signals, which vanish rapidly underwater, and the use of cameras is limited by the low visibility.

The case for deploying multiple robots rather than a single AUV rests on a coverage and observability argument: many cheap, decentralized units relying on local communication can sweep a search area far faster and with far more redundancy against individual failures than a single, more capable vehicle operating alone [13], a principle that has motivated recent work coupling swarm intelligence with digital twins for marine ecological monitoring [14] and bio-inspired collectives that coordinate three-dimensional behaviors using only implicit, locally sensed cues [15]. The underlying task, locating a region of the water column that matches an unknown target signature, also falls within the broader class of adaptive sampling and source-seeking problems, addressed for single vehicles tracking river plume fronts [16], sampling static marine fields [17], and tracing chemical plumes to their source [18]. Extending this task class to a swarm setting, where a target must be found cooperatively rather than

by a lone vehicle, is precisely the gap this thesis addresses.

Given the unknown and complex nature of the marine environment, AUVs need to be equipped with advanced sensors and algorithms to navigate and map the environment. To this end, Multi-agent Reinforcement Learning (MARL) algorithms have been recently proposed and increasingly used to overcome this task, enhancing the capabilities of Reinforcement Learning (RL) algorithms with swarm and cooperation capabilities, so that multiple agents can work together to achieve a common goal. A previous thesis [19] developed and analyzed a PPO-PSO algorithm to efficiently use swarm capabilities to overcome local optima and improve the performance of the PPO algorithm. The results showed that the PPO-PSO algorithm outperformed the standard PPO algorithm in terms of convergence speed and final performance, demonstrating the potential of MARL algorithms for complex tasks in unknown environments. However, the environment used in the previous thesis was a simplified simulation, and the necessity to test the algorithm in a more realistic environment is the main motivation for this thesis. Therefore, the main objective of this thesis is to evaluate the performance of a suitable MARL algorithm in a more realistic environment, where the agents are required to navigate and search for suitable coral restoration sites in a complex and unknown environment.

This thesis is organized as follows: Chapter 2 provides a comprehensive theoretical background on the Reinforcement Learning algorithms, including the multi-agent extension and general problem formulation. Chapter 3 describes the problem more specifically, including the assumptions, constraints and tools used. Chapter 4 introduces the proposed MARL algorithm, including its architecture, hyperparameters and implementation details. Chapter 5 presents the experimental setup, where multiple scenarios are created to test the capabilities of the obtained policy and its generalization. Finally, Chapter 6 concludes the thesis, summarizing the main findings, discussing the limitations and proposing future research directions.

Chapter 2

Theoretical Background

2.1 Reinforcement Learning

By definition, Reinforcement Learning (RL) algorithms learn solutions for sequential decision processes via repeated interaction with an environment [20]. An agent simply refers to a decision maker that receives information from the environment and takes actions based on that. The environment is defined as the external system with which the agent interacts, and may change state according to some transition dynamics, giving the agent a feedback (or reward). The agent's goal is to learn an optimal decision making policy that let him obtain the maximum cumulative reward over time. This optimal policy is obtained by trying different actions in different states and observing the outcomes, in a trial-and-error fashion.

Moreover, RL differs from other machine learning paradigms, like supervised or unsupervised learning, in that it does not require labeled input-output pairs, and the rewards act as a proxy from which to learn an optimal policy.

2.1.1 Markov Decision Process

Going into a more mathematical and formal definition, a sequential decision process can be modeled as a Markov Decision Process (MDP), which consists of:

- A set of states S ;
- A set of actions A ;
- A reward function $R : S \times A \times S \rightarrow \mathbb{R}$;
- A state transition probability function $P : S \times A \times S \rightarrow [0, 1]$ such that in state s taking action a the probability of transitioning to state s' is $p(s'|s, a)$;
- A policy $\pi : S \rightarrow A$ that maps states to actions, such that in state s the agent takes action a with probability $\pi(a|s)$;
- A discount factor $\gamma \in [0, 1]$ that determines the importance of future rewards.

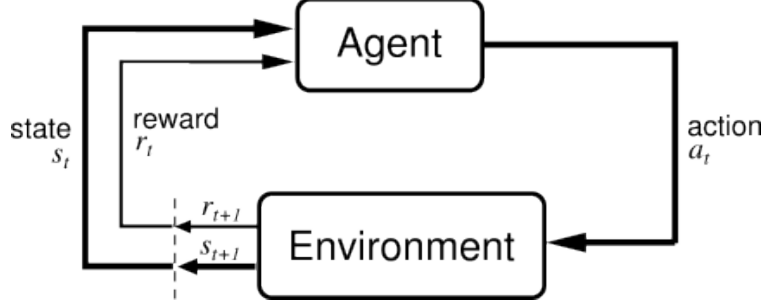


Figure 2.1: MDP framework.

An MDP starts in an initial state s_0 , sampled from the initial state distribution. At each time step t , the agent observes the current state s_t , takes an action a_t according to its policy $\pi(a_t|s_t)$ and receives a reward $r_t = R(s_t, a_t, s_{t+1})$, where s_{t+1} is the next state sampled from the transition probability distribution $p(s_{t+1}|s_t, a_t)$. Once the terminal state is reached, or after the maximum number of time steps, the episode ends. Figure 2.1 shows the described framework.

2.1.2 Value function

The Markov property states that the future state s_{t+1} depends only on the current state s_t and action a_t , and not on the previous states and actions.

$$Pr(s_{t+1}, r_t | s_t, a_t, s_{t-1}, a_{t-1}, \dots, s_0, a_0) = Pr(s_{t+1}, r_t | s_t, a_t) \quad (2.1)$$

Therefore, in an MDP, the expected return can be defined individually for each state $s \in S$, creating the concept of *value function*.

$$V_\pi(s) = \mathbb{E}_\pi [r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots | s_t = s] \quad (2.2)$$

$$= \sum_{a \in A} \pi(a|s) \sum_{s' \in S} \mathcal{P}(s'|s, a) [\mathcal{R}(s, a, s') + \gamma \mathbb{E}_\pi [u_{t+1} | s_{t+1} = s']] \quad (2.3)$$

$$= \sum_{a \in A} \pi(a|s) \sum_{s' \in S} \mathcal{P}(s'|s, a) [\mathcal{R}(s, a, s') + \gamma V_\pi(s')] \quad (2.4)$$

This is referred to as the Bellman equation, and it defines a system of m linear equations with m variables ($m = |S|$) and a unique solution, given by the value function V_π for policy π . If all elements of the MDP are known, the value function can be computed by solving the system of equations. However, in most real-world scenarios, the transition probabilities and reward functions are unknown, and the agent must learn the value function through interaction with the environment.

2.1.3 PPO

Value-based RL methods are a class of RL algorithms that learn a value function, which is used to estimate the expected return of a state or state-action pair. The value function

is usually represented as a neural network with parameters θ , and it is updated using the Bellman equation. The value function can be used to derive a policy by selecting the action that maximizes the expected return. The loss function is usually defined as squared error between the value function estimate and the target value y_t :

$$\mathcal{L}(\theta) = (y_t - Q(s_t, a_t; \theta)) \quad (2.5)$$

where y_t is the target value, defined as:

$$y_t = \begin{cases} r_t + \gamma \max_{a'} Q(s_{t+1}, a_{t+1}; \theta) & \text{if } s_{t+1} \text{ is not terminal} \\ r_t & \text{if } s_{t+1} \text{ is terminal} \end{cases} \quad (2.6)$$

Since the loss function contains the approximate value function twice, and we want to update the value estimate for the current state-action pair, the solution is to stop any gradient flow backward through the target, to avoid computing gradients through the bootstrapped target value. In fact, the bootstrapped value estimate should only serve as a target value to optimize the main value estimate $Q(s_t, a_t; \theta)$ toward.

To reduce the instability of training caused by the bootstrapped target value, the target network is used, which is a different neural network with parameters ϕ that is used to compute bootstrapped target values. Its parameters are periodically updated by copying the current parameters of the main value function network to the target network. Another problem is the correlation of consecutive experiences. The update of value function parameters with most recent experiences may lead to the agent forgetting optimal policies learned in the past and in other experiences. This is due to the fact the changes in the policy will lead to changes in the data distribution encountered by the agent, which then lead to changes in the optimal policy. The usual solution is to randomize the experience samples used to train the agent, that is why the replay buffer is used. In fact, a batch of experiences is sampled uniformly at random from the replay buffer, which will be used to update the value function parameters. This way, experiences can be reused multiple times (which leads to more efficient sampling) and gradients are more stable and with lower variance.

Policy gradient methods, instead, are a class of RL algorithms that learn a parameterized policy directly, without the need for a value function. The policy is usually represented as a neural network with parameters ϕ , and it is updated using the policy gradient theorem. It provides a theoretically founded expression for the gradient of the performance of a parameterized policy with respect to the parameters of the policy.

$$\nabla_{\phi} J(\phi) \propto \sum_{s \in \mathcal{S}} Pr(s|\pi) \sum_{a \in \mathcal{A}} Q^{\pi}(s, a) \nabla_{\phi} \pi(a|s; \phi) \quad (2.7)$$

where J represents a function measuring the quality of a policy π . It is similar to a loss function, but it is maximized and not minimized.

$$J(\phi) = \mathbf{E}_{\pi} \left[\sum_k \gamma^k r_{t+k} \right] \quad (2.8)$$

$$= \sum_s Pr(s'|s, a) \sum_a \pi(a|s; \phi) [R_t + \gamma V_{\pi}(s_{t+1})] \quad (2.9)$$

Then, by writing the policy gradient as an expectation with respect to the probability of states occurring under the current policy and the probability of actions being selected by the policy:

$$\nabla_{\phi} J(\phi) = \mathbf{E}_{s \sim Pr(\cdot|\pi), a \sim \pi(\cdot|s)} [Q^{\pi}(s, a) \nabla_{\phi} \log \pi(a|s; \phi)] \quad (2.10)$$

This illustrates how the optimization of the parameterized policy is limited to on-policy only data.

The expectation in the policy gradient can be either approximated or obtained from samples. REINFORCE uses Monte Carlo estimation as a sampling method, which uses on-policy samples of episodic returns to approximate the expected returns of the policy. It minimizes the following loss function:

$$\mathcal{L}(\phi) = -\frac{1}{T} \sum_{t=0}^{T-1} \left(\sum_{\tau=t}^{T-1} \gamma^{\tau-t} r^{\tau} \right) \log \pi(a^t|s^t; \phi) \quad (2.11)$$

Each episode, a full trajectory is collected by using current policy π , and then the return estimate is computed to estimate the policy gradient. However, Monte Carlo return estimates come with high variance and unstable training. Usually, the subtraction of a baseline (which keeps the bias but reduces the variance) is used as a solution.

Actor-critic algorithms train a parameterized policy (the actor) and a value function (the critic), alongside each other. The actor is optimized using gradient estimates derived from the policy gradient theorem, while the critic is used to compute bootstrapped return estimates. These estimates allow to estimate episodic returns just from the experience of a single step:

$$\mathbf{E}_{s^t \sim Pr(\cdot|\pi)} [u(h^t)|s^t] = \mathbf{E}_{s^t \sim Pr(\cdot|\pi), a^t \sim \pi(\cdot|s^t), s^{t+1} \sim \mathcal{T}(\cdot|s^t, a^t)} [\mathcal{R}(s^t, a^t, s^{t+1}) + \gamma V(s^{t+1})|s^t] \quad (2.12)$$

Moreover, bootstrapped return estimates exhibit lower variance compared to the Monte Carlo estimates of episodic returns, with the cost of introduced bias, since the value function might not approximate the true expected returns of states. Essentially, a trade-off between bias and variance has to be made, and intermediate solutions are available just by estimating N -step returns, aggregating the received rewards of N consecutive steps before computing a value estimate of the last state.

Advantages of the current policy can be computed to guide the policy gradients:

$$Adv^{\pi}(s, a) = Q^{\pi}(s, a) - V^{\pi}(s) \quad (2.13)$$

An advantage represents how much better or worse an action is compared to the average action in a given state. For a positive value, then, the probability of the policy π choosing that action should be increased, while for a negative value it should be decreased. The advantage function can be estimated using the critic's value function as:

$$Adv(s^t, a^t) = Q(s^t, a^t) - V(s^t) = \begin{cases} r^t + V(s^t) & \text{if } s^{t+1} \text{ is terminal} \\ r^t + \gamma V(s^{t+1}) - V(s^t) & \text{otherwise} \end{cases} \quad (2.14)$$

Therefore, the actor loss to be minimized becomes:

$$\mathcal{L}(\phi) = -Adv(s^t, a^t) \log \pi(a^t|s^t; \phi) \quad (2.15)$$

And the critic loss is defined as:

$$\mathcal{L}(\theta) = (y^t - V(s^t; \theta))^2 \quad (2.16)$$

Furthermore, entropy regularization can be added to the actor loss to discourage premature convergence to a suboptimal policy and incentivize exploration. It is a measure of the policy’s uncertainty, and minimizing the negative entropy penalizes it for assigning a very high probability to any action.

$$-\mathcal{H}(\pi(\cdot|s; \phi)) = \sum_{a \in \mathcal{A}} \pi(a|s; \phi) \log \pi(a|s; \phi) \quad (2.17)$$

Since the individual gradient update steps could lead to large policy updates, the idea of Trust Regions was introduced in TRPO [21], which essentially constrain the policy updates to a small area within the space of policy parameters in which the policy would not change significantly. PPO, instead, computes a computationally efficient surrogate objective to avoid large jumps in policy optimization steps. It makes use of importance sampling weights $\rho(s, a)$, which are defined as the fraction of probabilities of selecting a given action a in a state s for two policies:

$$\rho(s, a) = \frac{\pi(a|s; \phi)}{\pi_\beta(a|s)} \quad (2.18)$$

This factor adjusts the data distribution to make the data generated by π_β appear on-policy for π . Using these weights, PPO is able to update the policy multiple times using the same data and restrict the change of the policy by restricting the importance sampling weight. In fact, the actor loss becomes:

$$\mathcal{L}(\phi) = -\min(\rho(s^t, a^t) Adv(s^t, a^t), \text{clip}(\rho(s^t, a^t), 1 - \epsilon, 1 + \epsilon) Adv(s^t, a^t)) \quad (2.19)$$

Algorithm 1 summarize the whole procedure.

2.2 Multi-agent Reinforcement Learning

When multiple agents are taken into consideration, the concept of Multi-agent Reinforcement Learning (MARL) comes into play. In MARL, multiple agents interact with the same environment at the same time, and their actions can affect each other’s rewards and state transitions. Moreover, each agent has its own long-term reward to optimize, which now becomes a function of the policies of all other agents [22].

2.2.1 Markov Games

Markov Games (MGs) are a generalization of MDPs to the multi-agent setting, and are defined by a tuple $(\mathcal{N}, \mathcal{S}, \{\mathcal{A}^i\}_{i \in \mathcal{N}}, \mathcal{P}, \{\mathcal{R}^i\}_{i \in \mathcal{N}}, \gamma)$, where $\mathcal{N} = \{1, \dots, N\}$ denotes the set of agents, $\mathcal{S}$ denotes the state space observed by all agents, $\mathcal{A}^i$ denotes the action space of agent i . Let $A := A^1 \times \dots \times A^N$ denote the joint action space, then $\mathcal{P} : \mathcal{S} \times \mathcal{A} \rightarrow \delta(\mathcal{S})$ denotes the transition probability function from any state $s \in \mathcal{S}$ to any state $s' \in \mathcal{S}$ for any joint action

Algorithm 1 PPO

```

1: Initialize actor network  $\pi$  with random parameters  $\phi$ 
2: Initialize critic network  $V$  with random parameters  $\theta$ 
3: Repeat for every episode:
4:   for time step  $t = 0, 1, 2, \dots$  do
5:     Observe current state  $s^t$ 
6:     Sample action  $a^t \sim \pi(\cdot|s^t; \phi)$ 
7:     Apply action  $a^t$ ; observe reward  $r^t$  and next state  $s^{t+1}$ 
8:      $\pi_\beta(a^t|s^t) \leftarrow \pi(a^t|s^t; \phi)$   $\triangleright$  store the old policy probability
9:     for epoch  $e = 1, \dots, N_e$  do
10:       $\rho(s^t, a^t) \leftarrow \frac{\pi(a^t|s^t; \phi)}{\pi_\beta(a^t|s^t)}$ 
11:      if  $s^{t+1}$  is terminal then
12:        Advantage  $Adv(s^t, a^t) \leftarrow r^t - V(s^t; \theta)$ 
13:        Critic target  $y^t \leftarrow r^t$ 
14:      else
15:        Advantage  $Adv(s^t, a^t) \leftarrow r^t + \gamma V(s^{t+1}; \theta) - V(s^t; \theta)$ 
16:        Critic target  $y^t \leftarrow r^t + \gamma V(s^{t+1}; \theta)$ 
17:        Actor loss  $\mathcal{L}(\phi) \leftarrow -\min(\rho(s^t, a^t)Adv(s^t, a^t), \text{clip}(\rho(s^t, a^t), 1 - \epsilon, 1 + \epsilon)Adv(s^t, a^t))$ 
18:        Critic loss  $\mathcal{L}(\theta) \leftarrow (y^t - V(s^t; \theta))^2$ 
19:        Update parameters  $\phi$  by minimizing the actor loss  $\mathcal{L}(\phi)$ 
20:        Update parameters  $\theta$  by minimizing the critic loss  $\mathcal{L}(\theta)$ 

```

$a \in \mathcal{A}$; $R^i : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow \mathbb{R}$ is the reward function that determines the immediate reward received by agent i for a transition from (s, a) to s' ; $\gamma \in [0, 1)$ is the discount factor.

At each time step t , each agent executes an action a_t^i according to the system state s_t . The system transitions to a new state s_{t+1} and gives each agent a reward r_t^i . Each agent wants to maximize its long-term reward, finding a policy π^i . As a consequence, the value-function $V^i : \mathcal{S} \rightarrow \mathbb{R}$ of agent i becomes a function of the joint policy π defined as $\pi(a|s) = \prod_{i \in \mathcal{N}} \pi^i(a^i|s)$.

$$V_{\pi^i, \pi^{-i}}^i := \mathbb{E} \left[\sum_{t \geq 0} \gamma^t R^i(s_t, a_t, s_{t+1}) | a_t^i \sim \pi^i(\cdot|s_t), s_0 = s \right] \quad (2.20)$$

Since each agent's value function depends on the policies of all other agents, the optimal policy for each agent is defined as a Nash equilibrium, where no agent can improve its long-term reward by unilaterally changing its policy. A Nash equilibrium is a joint policy $\pi^* = (\pi^{1*}, \dots, \pi^{N*})$ such that for all agents $i \in \mathcal{N}$ and for all states $s \in \mathcal{S}$, we have:

$$V_{\pi^{i,*}, \pi^{-i,*}}^i(s) \geq V_{\pi^i, \pi^{-i,*}}^i(s), \quad \forall \pi^i \quad (2.21)$$

In other words, at a Nash equilibrium, each agent's policy is the best response to the policies of all other agents, and no agent can improve its long-term reward by changing its policy while the others keep theirs fixed. It always exists for finite-space infinite-horizon discounted MGs, but may not be unique in general.

2.2.2 Cooperative setting

In some tasks, agent may need to cooperate to achieve a common goal, and in these cases they share a common reward function, i.e., $R^1 = R^2 = \dots = R^N = R$. With this model in mind, the value function and Q-function are identical to all agents, which thus enables the single-agent RL algorithms to be applied, if all agents are coordinated as one decision maker. Moreover, agents are allowed to have different reward functions, which may be kept private to each agent, while the goal for cooperation is to optimize the long-term reward corresponding to the average reward $\bar{R}(s, a, s') := N^{-1} \cdot \sum_{i \in \mathcal{N}} R^i(s, a, s')$ for any $(s, a, s') \in \mathcal{S} \times \mathcal{A} \times \mathcal{A}$.

2.2.3 Challenges of MARL

MARL algorithms may be useful in many real-world applications, but they face several challenges that make them more difficult to design and analyze than single-agent RL algorithms.

- **Non-stationarity:** since multiple agents learn concurrently, the environment faced by each agent is non-stationary, since the action taken by one agent affects the reward of other opponent agents, and the evolution of the state. As a result, each agent needs to learn to account for how other agents behave and adapt to the joint behavior accordingly. In fact, if each agent ignores this issue and optimizes its own policy assuming a stationary environment (*independent learning*), the algorithms may fail to converge.
- **Non-unique learning goals:** the learning goals in multi-agent settings may be unclear or conflicting, as each agent may have its own objectives and preferences, which may not align with those of other agents. This can lead to situations where agents are competing for resources or trying to achieve different goals, making it difficult to design algorithms that can effectively balance the needs of all agents.
- **Credit assignment problem:** in multi-agent settings, it can be difficult to determine which agent's actions contributed to the overall outcome, especially when the agents are working together towards a common goal. This can make it challenging to assign credit or blame for the success or failure of the team, and can lead to suboptimal learning outcomes.
- **Scalability:** to handle non-stationarity, each individual agent may need to account for the joint action space, whose dimension increases exponentially with the number of agents. This can make it difficult to scale MARL algorithms to large numbers of agents, as the computational and communication requirements may become prohibitive.
- **Partial observability:** in many real-world scenarios, agents may not have access to the full state of the environment, and may only be able to observe a limited subset of the state variables. This can make it difficult for agents to learn effective policies, as they may not have enough information to make informed decisions.

Chapter 3

Problem formulation

A previous thesis [19] already developed and analyzed a MARL algorithm suitable for this task. It used a gaussian envelope and a multifrequency cosine landscape to define the reward function. The real world, however, is more complex and made of currents. This thesis, thus, brings these contributions:

- A new, more realistic environment, composed of currents, salinity and turbidity fields, created with Oceananigans;
- Another dimension to the problem, so that it goes from 2D to 3D;
- The use of a simulator, SwarmSwIM, to connect the environment with the agents and the MARL algorithm.

In the following sections, all those components are described in detail, including the parameters used to create the environment with the solver and how agents are commanded via the simulator.

3.1 SwarmSwIM Simulator

SwarmSwIM [23] is a simulator developed to help researchers and practitioners to evaluate and enhance the control systems and decision-making capabilities of underwater agents (AUVs). Completely Python-based, it provides tools for current modeling, agent control and visual representation, balancing computational efficiency with sufficient environmental fidelity to facilitate large-scale swarm simulations. The low-level control of the agents is approximated by the closed-loop control response of each agent, such as depth and heading keeping, in the resolution of the agent motion.

Each member of the swarm is an *agent* and is represented by a point in the 3D space defined by the domain of the environment. The main cycle of it is made by simulation and agent ticks, where the former refers to the global update while the latter to the local update of each agent.

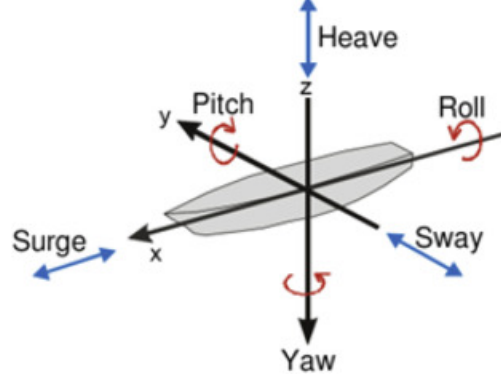


Figure 3.1: Representation of the 6 DoF of an AUV [24].

3.2 Agent

An agent is a point in the 3D space and its state is determined by four DoF: the 3D position and the yaw angle. Roll and pitch are not considered, and yaw is fixed during an episode for simplicity. Agent control is achieved by three schemes, as showed in Figure 3.1:

1. *Depth*: Set to heave, such that the vertical velocity is taken as input;

$$z_{t+1} = (v_z + \epsilon)\Delta t + z_t \quad \text{with } \epsilon \sim \mathcal{N}(0, 1) \quad (3.1)$$

2. *Heading*: Set to yawrate but the yaw is always fixed;

$$\psi_{t+1} = (v_\psi + \epsilon)\Delta t + \psi_t \quad (3.2)$$

3. *Planar*: Set to local velocity, such that current position is updated based on a velocity input command expressed in the body-frame reference system.

$$\begin{bmatrix} x_{t+1} \\ y_{t+1} \end{bmatrix} = \begin{bmatrix} v_x + \epsilon \\ v_y + \epsilon \end{bmatrix} \Delta t + \begin{bmatrix} x_t \\ y_t \end{bmatrix} \quad (3.3)$$

In this work, the action space is a discrete set of 27 actions, which corresponds to the 26 neighbors of a cell in a 3D grid plus the "stay" action. Thus, each action corresponds to a movement in the continuous space that is discretized by the grid, and is splitted into the three components as described above. The total magnitude of the velocity is fixed to $1m/s$, which is a reasonable value for an AUV. It follows that, given the total velocity and the angle of the action in the horizontal and vertical dimensions, the three components of the velocity are computed as follows:

$$v_x = v \cos(\theta) \cos(\phi) \quad v_y = v \cos(\theta) \sin(\phi) \quad v_z = v \sin(\theta) \quad (3.4)$$

where θ is the angle in the vertical plane and ϕ is the angle in the horizontal plane. The angles are computed based on the action index, which is a number between 0 and 26. Figure 3.2 illustrates this.

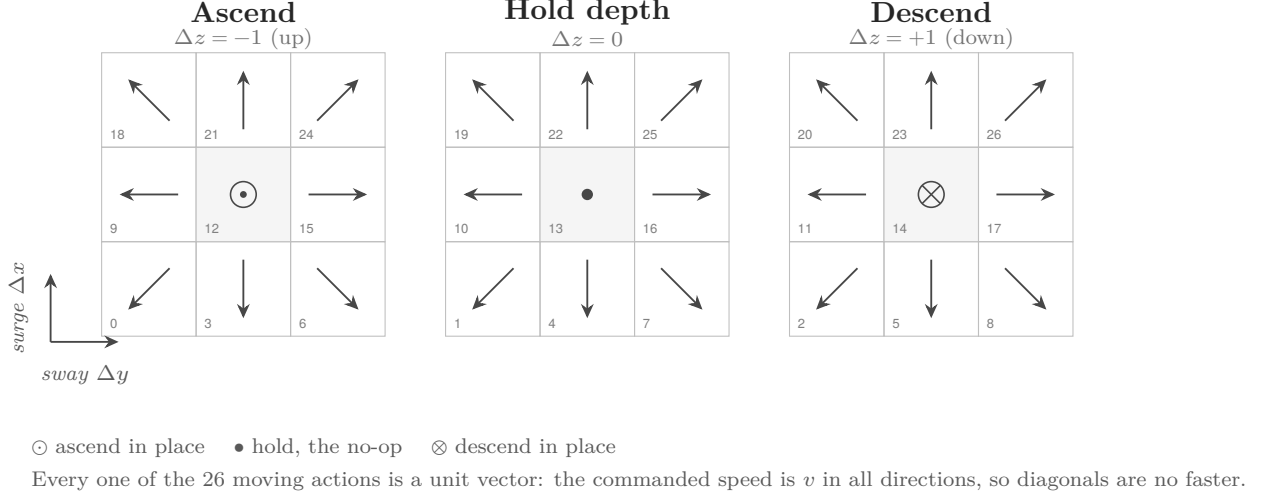


Figure 3.2: The 27 discrete actions, decomposed by the vertical component Δz into three horizontal keypads. Each cell shows the commanded direction and its action index $a = 9(\Delta x + 1) + 3(\Delta y + 1) + (\Delta z + 1)$, where Δx is surge and Δy is sway in the body frame. Because z is measured positive downward, $\Delta z = +1$ descends. The centre cell of each keypad is one of the three purely vertical actions, with $a = 13$ the no-op. Normalising every triple to unit length makes the commanded speed v identical in all 26 directions, so diagonal moves cover no more ground per step than axis-aligned ones.

3.3 Environment

The environment is a 3D domain where the agents operate. To make it more realistic, I used Oceananigans.jl [25], which is a fast, friendly and flexible software package for finite volume simulations of the nonhydrostatic and hydrostatic Boussinesq equations on CPUs and GPUs. An environment used in an episode is simply a snapshot of one of the different simulations I ran with Oceananigans. The environment is composed of three fields:

- *Currents*
- *Salinity*
- *Turbidity*

The first one is computed by the solver. The second one is computed using N salinity sources, initialized randomly at "land" positions (i.e., south and west walls in this case) and with a random strength. The field is constructed by advecting the particles with the current field, corrected by a diffusion term. The third is simply an analytical function of depth:

$$T(z) = \frac{1}{1 + e^{-(z-z_0)}} \quad \text{with } z_0 = 0.5 \quad (3.5)$$

The domain is a cube of $1000 \times 1000 \times 100$ meters, discretized on a $128 \times 128 \times 64$ grid of cells, with topology (Periodic, Periodic, Bounded). The method chosen for the simulation

Table 3.1: Parameters used in the Oceananigans nonhydrostatic model

Parameter	Value	Description
LX	1000.0	Domain (x) [m]
LY	1000.0	Domain (y) [m]
LZ	100.0	Domain (z) [m]
NX	128	Grid points (x) [m]
NY	128	Grid points (y) [m]
NZ	64	Grid points (z) [m]
$T_{surface}$	22.0 / 33.0	Surface temperature (winter/summer) [$^{\circ}C$]
T_{bottom}	20.0 / 22.0	Bottom temperature (winter/summer) [$^{\circ}C$]
$S_{baseline}$	40.0 / 42.0	Baseline salinity (winter/summer) [PSU]
V_{wind}	5.0 / 8.0	Wind speed (winter/summer) [m/s]
ρ_{air}	1.225	Air density [kg/m^3]
c_{drag}	$1.3e^{-3}$	Drag coefficient
ρ_{water}	1027.0	Water density [kg/m^3]
Latitude	24.5	Abu Dhabi latitude (degrees N)
ω_{earth}	$7.292e^{-5}$	Earth rotation rate [rad/s]
$f_{coriolis}$	$2\omega_{earth} \sin(\text{Latitude})$	Coriolis parameter [rad/s]
α_t	$1.67e^{-4}$	Thermal expansion coefficient [K^{-1}]
β_s	0	Salinity contraction coefficient [psu^{-1}]
σ_h	300.0	Salinity horizontal std [m]
σ_v	40.0	Salinity vertical std [m]
Salinity excess	10.0	Excess over baseline [PSU]
Salinity decay time	1.0	Time to relax the field to a gaussian [h]
Warmup time	12.0	Time to reach a steady state [h]
Simulation time	6.0	Total simulation time [h]
Output frequency	12.0	Frequency of output [min]

is the nonhydrostatic one, which assumes that the vertical acceleration is not negligible. This method is more accurate than the hydrostatic one, but necessitates more computational resources. 40 simulations were carried out for training and 10 for testing, each one with a different random seed to ensure variability in the environment. Each simulation created a single NetCDF file, with a snapshot containing the fields every 12 minutes. The simulation was run for 8 hours, with the first 4 used as warmup time and so not recorded.

3.3.1 Currents

Currents are composed by three components: the horizontal velocities (u, v), and the vertical one (w). They are prognostic variables integrated forward in time by the nonhydrostatic Oceananigans model, which solves the rotating, Boussinesq, nonhydrostatic Navier–Stokes equations with an LES subgrid closure. The developed script only sets up the forcing, the boundary conditions, the rotation, the stratification, and the time-stepping; the flow itself emerges from the solve. The model integrates the following equation:

$$\frac{\partial \vec{\mathbf{u}}}{\partial t} + \underbrace{(\vec{\mathbf{u}} \cdot \nabla) \vec{\mathbf{u}}}_{\text{advection}} + \underbrace{f \hat{\mathbf{z}} \times \vec{\mathbf{u}}}_{\text{coriolis}} = - \underbrace{\nabla p}_{\text{pressure}} + \underbrace{b \hat{\mathbf{z}}}_{\text{buoyancy}} + \underbrace{\nabla \cdot \tau_{\text{sgs}}}_{\text{subgrid stress}} + \underbrace{\mathbf{F}}_{\text{body force}} \quad (3.6)$$

$$\text{with } \nabla \cdot \vec{\mathbf{u}} = 0$$

The second term $((\vec{\mathbf{u}} \cdot \nabla) \vec{\mathbf{u}})$ is the advection term. It is not a force, but refers to the velocity at a fixed grid cell. This is the term that makes the Navier-Stokes equations nonlinear, and it is responsible for the transfer of energy from large to small scales, which is the essence of turbulence. In other words, it is the term that makes the flow chaotic and unpredictable.

The third term $(f \hat{\mathbf{z}} \times \vec{\mathbf{u}})$ is the Coriolis force. It is a global operator (i.e., modifies the entire domain) and is a vector perpendicular to the velocity, with magnitude proportional to it.

The fourth term $(-\nabla p)$ is the pressure gradient force. It is a global operator either and essentially is a vector pointing in the direction of steepest increase in pressure. Because the model is Boussinesq, p here is the kinematic pressure, already divided by $\rho_0 = 1027.0$. Moreover, it has no equation of state, it is a Lagrange multiplier that enforces incompressibility, and since there is the constraint $\nabla \cdot \vec{\mathbf{u}} = 0$, it is whatever force field required to make that true. Mechanically, Oceananigans does this with a projection method each timestep:

- advances velocity ignoring pressure
- solves a Poisson equation for pressure

$$\nabla^2 p = \nabla \cdot \vec{\mathbf{u}}^* / \Delta t \quad (3.7)$$

- corrects the velocity with the pressure gradient

$$\vec{\mathbf{u}} = \vec{\mathbf{u}}^* - \Delta t \cdot \nabla p \quad (3.8)$$

The second step is an elliptic solve that uses an FFT along the periodic axis and transforms along the bounded axes.

The fifth term $(b \hat{\mathbf{z}})$ is the buoyancy force, which is a vertical force per unit of mass. It is a function of the density anomaly $\rho' = \rho - \rho_0$, where ρ_0 is the reference density.

$$b = -g \frac{\rho'}{\rho_0} = g (\alpha(T - T_0) - \beta(S - S_0)) \quad (3.9)$$

since ρ' is supplied as a linear function of temperature and salinity.

The sixth term $(\nabla \cdot \tau_{\text{sgs}})$ is the subgrid stress, which is a closure for the unresolved scales of motion. Essentially, LES applies a spatial filter (the grid) to the Navier-Stokes equations, splitting the velocity into a resolved and an unresolved part. Physically, τ_{sgs} is the momentum flux carried by eddies smaller than a grid cell, since in the current model the grid cells cannot see them.

The last term $(\mathbf{F})$ embodies every forcing that is not in the standard list of fixed physics solved by Oceananigans. It is a body force, applied throughout the volume at every grid cell.

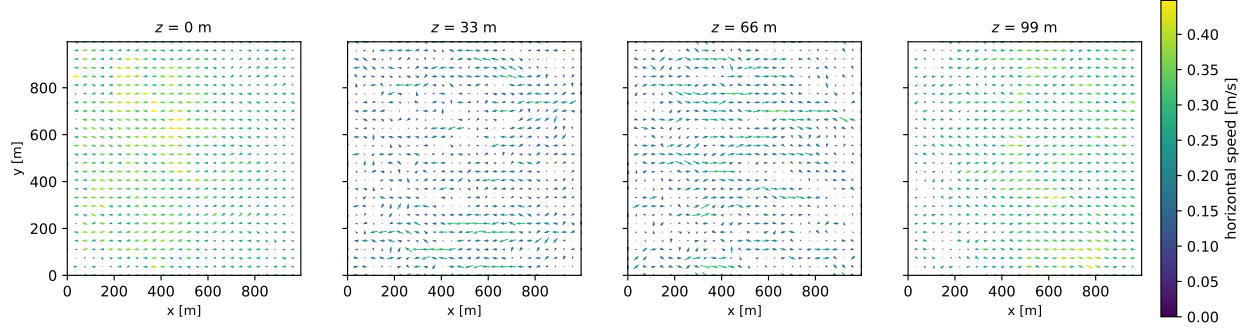


Figure 3.3: Currents field at four different depths, with the color representing the magnitude of the velocity. The arrows represent the direction of the current.

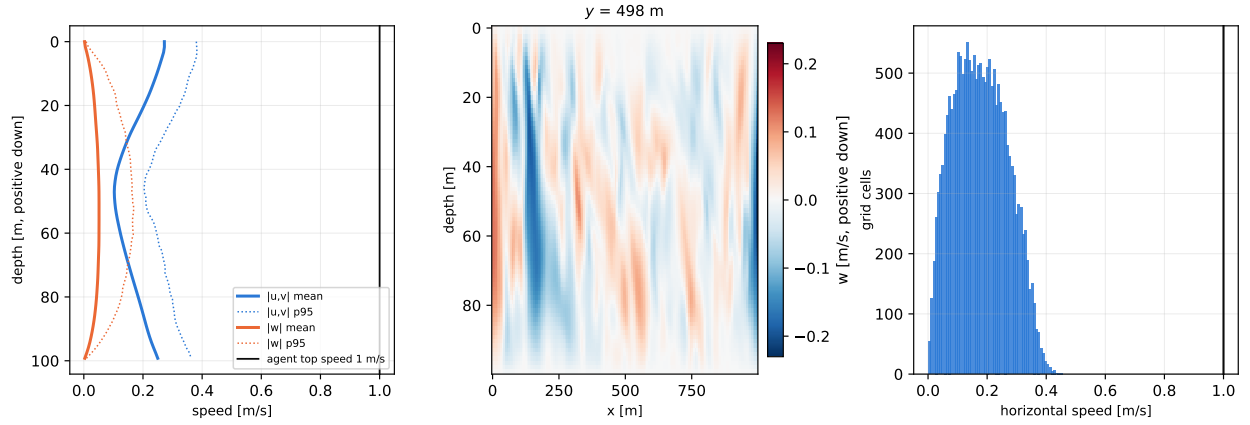


Figure 3.4: Currents statistics. The first plot shows the aggregate horizontal velocity magnitude distribution at varying depth. The second plot shows the vertical current field magnitude at fixed y , varying x and depth. The third plot shows the overall horizontal magnitude distribution. The dark line refers to the agent's speed.

Wind speed enters whrough the bulk drag law:

$$\tau = \rho_{\text{air}} C_d U_{10}^2 \implies \tau_{\text{kin}} = \frac{\rho_{\text{air}} C_d U_{10}^2}{\rho_{\text{water}}} \quad (3.10)$$

And that is applied as a flux boundary condition at the surface. Momentum enters through the surface and is transferred downward by turbulence. Surface stress plus rotation gives the classical Ekman spiral: the current at the surface is deflected to the right of the wind (northern hemisphere) and rotates further right with depth while decaying.

The resulting field is shown in Figure 3.3. In the selected file, the horizontal currents' magnitude has an average of 0.1784m/s , with a maximum value of 0.4545m/s . The vertical one, instead, has an average of 0.0382m/s and a maximum of 0.1691m/s , as can be seen in Figure 3.4.

Table 3.2: Overall currents' statistics in the training simulations

Quantity	Mean	Median	p95	p99	Max
$ u, v $ horizontal	0.1940	0.1915	0.3549	0.4051	0.7266
$ w $ vertical	0.0356	0.0241	0.1105	0.1709	0.4627

Finally, Table 3.2 shows the overall statistics of the currents field across all the simulations used for training.

3.3.2 Salinity Field

Salinity is the second field of the environment, and it is the variable that alongside the turbidity defines the target zone for the agents. In this work it is computed by advecting N salinity sources, initialized randomly at south and west walls at surface and with a random strength. At each timestep, the sources emit salinity particles, which are advected by the currents and corrected by a diffusion term that bounds the PSU range to 10 PSU. That was a choice made either to have a navigable field with computable gradients and to resemble the realistic salinity range of the Gulf.

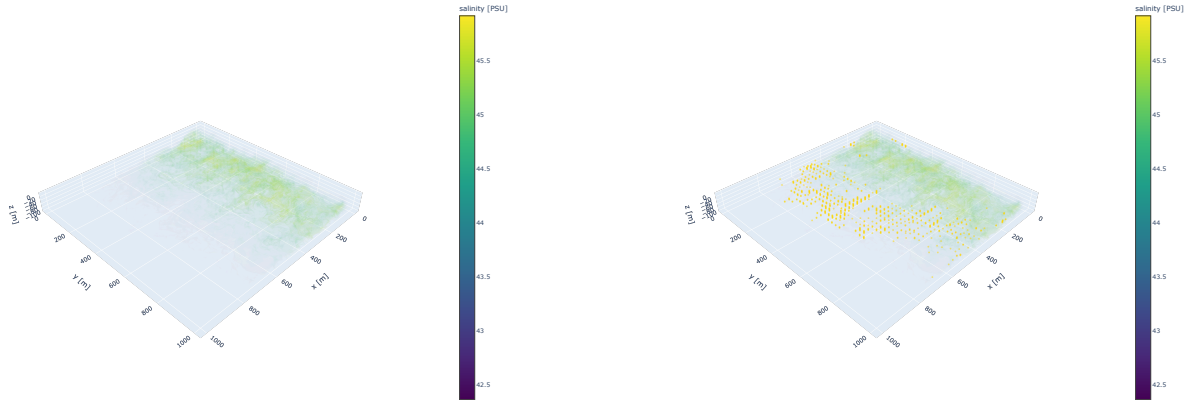


Figure 3.5: Salinity field of a simulation. The left plot shows the salinity field in the 3D environment, with the color representing the PSU value. The right plot adds also the target zone, defined as the points that satisfy a salinity value of 43.852 ± 0.15 and a turbidity value of 0.4194 ± 0.05 .

As can be seen in Figure 3.5, the PSU range is between 41.974 and 45.913, so the resulting PSU range is not exactly 10 PSU, but it is still navigable and the gradients are informative for the agents. The target zone is defined as the points that satisfy a given salinity and turbidity value, which are obtained from a sampled point in the environment at training time, to ensure that the target zone is always reachable. Moreover, the default errors for the two fields are set to 0.15 and 0.05, respectively. Table 3.3 shows the overall statistics of the salinity field across all the simulations used for training.

Table 3.3: Overall salinity statistics in the training simulations

Quantity	Mean	Median	p95	p99	Max	Min
S [PSU]	43.678	43.699	45.309	45.635	46.935	39.784
$ \nabla_{x,y} S $	0.01571	0.01038	0.04825	0.07688	0.29711	0.0
$ \nabla_z S $	0.02786	0.01469	0.10238	0.17587	0.73736	0.0

Chapter 4

Proposed Method

The task at hand can be described as target area search and navigation in an unknown environment. It is an active research field, even though the specific task varies. In fact, different authors have considered using MARL solutions to target localization, path planning and source seeking problems, which are all related to the problem at hand but with different objectives. Granadeno et al. [26] focuses on multi-source plume tracing, a task where a team of agents is required to find the source of a chemical plume in a 3D environment. The authors used a MARL approach, with cooperative agents and CTDE paradigm. They found that their approach outperformed other methods based on non-action dependent networks, and with a higher efficiency. Moreover, they found out that by increasing the number of cooperative agents, the success rate improves. Other works, such as [27], [28], [29], [30], [31] and [32], tackle the problem of path planning using hybrid approaches based on Q-learning and Particle Swarm Optimization (PSO), the former being another RL algorithm based on action-value function and the latter being a population-based optimization algorithm that simulates the social behavior of birds and fishes, i.e. the velocity vector of each particle is updated based on its own best position and the best position of the swarm. It is highly effective with large swarms, where the effect of the social component is more pronounced and lets the swarm converge faster and more efficiently. Finally, other works such as [33], [34] and [35] explore the use of MARL approaches in the task of target search and localization in unknown environments, in the shortest possible time. Here, bayesian approaches and algorithms like MADDPG or QMIX are used, always with the CTDE paradigm, and the authors found that their approaches outperform other methods like Ant Colony Optimization (ACO) and Deep Q-Networks (DQN), with higher efficiency.

The novelty of this work, in fact, is the use of MARL algorithms to search a specific target area, specified by some parameters that in this case are the turbidity and the salinity, and the creation of a realistic 3D environment with Oceananigans. To do so, IPPO [36] and MAPPO [37] have been chosen as first approaches to solve the problem, given their popularity and effectiveness in multi-agent scenarios. The following sections will describe them in detail.

4.1 IPPO

Independent Proximal Policy Optimization (IPPO) is the multi-agent version of the popular Proximal Policy Optimization (PPO) algorithm [38], an on-policy RL algorithm that uses a clipped surrogate objective to update the policy. In IPPO, each agent is used to learn a common policy, so the agents share its parameters. It is like having N independent agents each with the same PPO-trained policy on a single-agent scenario, but the use of shared parameters helps having a more efficient sampling since in an episode multiple gradients are computed and used from different parts of the environment.

The problem is formulated as a Decentralized Partially Observable Markov Decision Process (Dec-POMDP), in a cooperative setting. Formally, it is defined by a tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{O}, \mathcal{R}, \mathcal{P}, n, \gamma \rangle$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the shared action space for each agent i , $\mathcal{O}$ is the local observation space (each o_i is the local observation for agent i at global state s), $\mathcal{P}(s'|s, A)$ is the transition probability from s to s' given the joint action A , $\mathcal{R}(s, A)$ is the shared reward function and γ is the discount factor. Agents use a policy $\pi_\theta(a_i|o_i)$ to produce an action from the local observation and jointly optimize the discounted accumulated reward $\mathcal{J}(\theta) = \mathbb{E}_{A^t, s^t} \left[\sum_t \gamma^t R(s^t, A^t) \right]$.

In IPPO, a variant of the advantage function is used, where each agent learns a local observation based critic $V_\phi(z_t^a)$ parameterised by ϕ using *Generalized Advantage Estimation* (GAE). For each agent a , the relative advantage function is defined as:

$$A_t^a = \sum_{l=0}^h (\gamma \lambda)^l \delta_{t+l}^a \quad (4.1)$$

where $\delta_t^a = r_t^a + \gamma V_\phi(z_{t+1}^a) - V_\phi(z_t^a)$ is the temporal difference error, γ is the discount factor and λ is a hyperparameter that controls the bias-variance trade-off of the advantage estimates. The final policy loss for each agent is defined as:

$$\mathcal{L}^a(\theta) = \mathbb{E}_{z_t^a, u_t^a} \left[\min \left(\frac{\pi_\theta(u_t^a|z_t^a)}{\pi_{\theta_{old}}(u_t^a|z_t^a)} A_t^a, \text{clip} \left(\frac{\pi_\theta(u_t^a|z_t^a)}{\pi_{\theta_{old}}(u_t^a|z_t^a)}, 1 - \epsilon, 1 + \epsilon \right) A_t^a \right) \right] \quad (4.2)$$

The chosen policies are two simple MLPs with two hidden layers of 256 units for the actor and 128 for the critic. Moreover, multiple environments are stepped in parallel, so that each global step is composed of n agents in E environments. For a total number of timesteps, the relative number of iterations is given by $\frac{\text{total_timesteps}}{n \cdot E \cdot \text{max_steps}}$. In each iteration, a number of updates given by update_epochs is performed, with a minibatch size equal to $\frac{\text{max_steps}}{\text{num_minibatches}}$. Observations and returns are normalized, and the algorithm stops when the Kullback Leibler (KL) divergence between the current policy and the old one is higher than target_kl . The full procedure is summarized in Algorithm 2. A binary mask m_t^a excludes from the loss the steps of agents that have already terminated their episode, so that frozen agents do not contribute spurious gradients. The used hyperparameters are in Tables 4.1 and 4.2.

4.2 MAPPO

The policy π_θ and the value function V_ϕ are two separate neural networks, where the latter is used for variance reduction and only utilized during training. Hence, as explained in

Algorithm 2 IPPO

```
1: Initialize  $\theta, \phi$  using Orthogonal initialization
2: while  $step \leq step_{max}$  do
3:    $\theta_{old} \leftarrow \theta$   $\triangleright$  freeze the behaviour policy
4:   for  $t = 0, \dots, T - 1$  do  $\triangleright$  rollout phase
5:     for all agents  $a$  in each environment do
6:       sample  $a_t^i \sim \pi_{\theta_{old}}(a_t^i | o_t^i)$  and store  $\log \pi_{\theta_{old}}(a_t^i | o_t^i)$  and  $a_t^i$ 
7:       step the environments and observe  $r_t^i, o_{t+1}^i$  and the termination flags  $d_{t+1}^a$ 
8:       store the transition together with the mask  $m_t^a = 1 - d_t^a$ 
9:   for epoch  $k = 1, \dots, K$  do  $\triangleright$  update phase
10:    for  $t = T - 1, \dots, 0$  do  $\triangleright$  precompute adv with current critic each epoch
11:       $\delta_t^i \leftarrow r_t^i + \gamma(1 - d_{t+1}^i) V_\phi(o_{t+1}^i) - V_\phi(o_t^i)$ 
12:       $Adv_t^i \leftarrow \delta_t^i + \gamma\lambda(1 - d_{t+1}^i) Adv_{t+1}^i, \quad \hat{V}_t^i \leftarrow Adv_t^i + V_\phi(o_t^i)$ 
13:      for all minibatches  $b \subset \mathcal{B}$  of size  $|\mathcal{B}|/M$  do  $\triangleright$  gradient computation
14:        normalize the advantages  $A_t^a$  over  $b$  to zero mean and unit variance
15:         $\rho_t^i \leftarrow \pi_\theta(a_t^i | \tilde{o}_t^i) / \pi_{\theta_{old}}(a_t^i | \tilde{o}_t^i)$ 
16:         $\mathcal{L} \leftarrow -\langle \min(\rho Adv, \text{clip}(\rho, 1 \pm \epsilon) Adv) \rangle_m + c_v \langle \frac{1}{2} (V_\theta - \hat{V})^2 \rangle_m - c_e \langle \mathcal{H}[\pi_\theta] \rangle_m$ 
17:        update  $(\theta, \phi)$  by gradient descent on  $\mathcal{L}$  (Adam, gradient norm clipped)
18:      if  $\widehat{\text{KL}}(\pi_{\theta_{old}} \| \pi_\theta) > \kappa$  then
19:        break  $\triangleright$  early stopping on excessive policy change
```

[37], it can be centralized and given access to the global state s . This creates the CTDE paradigm, where the policy is decentralized and only uses local observations, while the critic is centralized and uses global information. This is the only thing that differentiates MAPPO from IPPO, as exposed in Algorithm 3. The hyperparameters are the same as in Table 4.2.

4.3 Agent State

In a RL framework, the agent state is the information that the agent receives from the environment at each timestep. It must be carefully designed to provide the agent with enough information to make it able to learn the optimal policy in the given environment. Moreover, it must take into consideration the real world constraints of the problem, such as the limited sensing capabilities of the agents. In this work, there is a *local state*, which is the information that each agent receives from the environment considering the fact that underwater vehicles don't know their global position, and a *global state*, which considers the global position of the agents and is used in the centralized critic of MAPPO. The local state is composed of the following information:

- $\mathbf{u}, \mathbf{v}, \mathbf{w}$, the components of the currents at the agent's position;
- $\frac{\partial S}{\partial u}, \frac{\partial S}{\partial v}, \frac{\partial S}{\partial w}$, the components of the salinity gradient at the agent's position. These could theoretically be computed by the agent using some sensors, so they were included in the state;

Table 4.1: Default env parameters

Parameter	Value
ϵ_S	0.15
ϵ_τ	0.05
σ_S	1.5
σ_τ	0.3
n_agents	2
k	12
v_agent	1.0
max_steps	1800
δt	0.1
frame_skip	10
success_bonus	20.0

- $S - S^*$, the difference between the salinity computed at the agent’s position and the target salinity;
- $\tau - \tau^*$, the difference between the turbidity computed at the agent’s position and the target turbidity;
- z , the depth of the agent, which is important to know since the turbidity is a function of it and it is measurable by the agent using a pressure sensor;
- *history* of the last k $(\Delta x, \Delta y, \Delta z, S - S^*, \tau - \tau^*)$ tuples, where $\Delta x, \Delta y, \Delta z$ are the relative displacements of the agent in the last k steps, that could be measured in real world from the agent by knowing its initial positions and the actions it does. This is important to give the agent a sense of direction and speed, since it doesn’t know its global position.

This is the minimum information that the agent needs to learn an optimal policy for this task, and specifically the history is crucial, since it helps achieving a higher success rate. The reason for this is probably that the agent can make a sort of global map of where it is going and where it has already been, improving the search efficiency and effectiveness. Moreover, a *communication* flag that simulates the exchange of information between agents in the whole domain was added, so that when it is active the agents can share their local displacement and target differences, i.e. in addition to the local state, each agent receives $N - 1$ tuples of $(\Delta x, \Delta y, \Delta z, S - S^*, \tau - \tau^*)$ from the other agents. This is important to improve the cooperation between agents and to avoid them searching in the same areas, i.e., division of labour.

The global state, instead, adds on top of the local state the global horizontal position of the agent, i.e. x, y . This is used in the centralized critic of MAPPO, which is only used during training and so can be feed with global information the agent does not have during execution. The final state space is a vector of 11 elements, against the $9 + 5 \cdot k(+5 \cdot (N - 1))$ elements of the local state.

Table 4.2: IPPO parameters

Parameter	Value
total_timesteps	10M
learning_rate	0.0003
num_envs	12
num_steps	512
anneal_lr	False
γ	0.9995
λ_{GAE}	0.95
<i>num_minibatches</i>	24
update_epochs	10
norm_adv	True
clip_coeff	0.25
clip_vloss	False
entropy_coef	0.01
anneal_entropy	True
ent_anneal_frac	0.5
ent_coef_final	0.0
vf_coef	0.5
max_grad_norm	0.5
target_kl	0.02

4.4 Reward Function

The reward function is crucial in RL, since it is the only feedback that the agent receives from the environment. Usually, a sparse reward is used, so that the agent receives a negative unitary reward at each timestep and a positive reward when it reaches the goal. However, this approach is not ideal in most cases, since the environment could be very large and complex, and the agent may reach the goal only few times during training, making it almost impossible a successful learning in that context. For that reason, a dense reward function is used in such cases. It is simply the construction of a function that reward the agent for going toward the goal, so that the closer it is to it, the higher the reward it receives. The difficulty is to find a correct function so that the agent is not incentivized to stay in a local minimum, i.e. a position where it receives a high reward but is not the goal, and has no incentives to deviate from that point. The idea is to use a *potential* value $\phi(s)$, associated with each state s , and add to the reward of a transition the difference of potentials [39]. Thus, a shaping reward function $\mathcal{F} : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow \mathbb{R}$ is potential-based if there exists $\phi : \mathcal{S} \rightarrow \mathbb{R}$ such that:

$$\mathcal{F}(s, a, s') = \gamma\phi(s') - \phi(s) \quad (4.3)$$

This is the concept of *potential reward shaping*, and is used in this work. If $\mathcal{F}$ is a potential-based function, then every optimal policy in $M' = \langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R} + \mathcal{F}, \gamma \rangle$ is also an optimal policy in $M = \langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$ and vice versa.

Algorithm 3 MAPPO

```
1: Initialize  $\theta, \phi$  using Orthogonal initialization
2: while  $step \leq step_{max}$  do
3:    $\theta_{old} \leftarrow \theta$   $\triangleright$  freeze the behaviour policy
4:   for  $t = 0, \dots, T - 1$  do  $\triangleright$  rollout phase
5:     for all agents  $a$  in each environment do
6:       sample  $a_t^i \sim \pi_{\theta_{old}}(a_t^i | o_t^i)$  and store  $\log \pi_{\theta_{old}}(a_t^i | o_t^i)$  and  $a_t^i$ 
7:       step the environments and observe  $r_t^i, o_{t+1}^i$  and the termination flags  $d_{t+1}^a$ 
8:       store the transition together with the mask  $m_t^a = 1 - d_t^a$ 
9:   for epoch  $k = 1, \dots, K$  do  $\triangleright$  update phase
10:    for  $t = T - 1, \dots, 0$  do  $\triangleright$  precompute adv with current critic each epoch
11:       $\delta_t^i \leftarrow r_t^i + \gamma(1 - d_{t+1}^i) V_\phi(s_{t+1}^i) - V_\phi(s_t^i)$ 
12:       $Adv_t^i \leftarrow \delta_t^i + \gamma\lambda(1 - d_{t+1}^i) Adv_{t+1}^i, \quad \hat{V}_t^i \leftarrow Adv_t^i + V_\phi(s_t^i)$ 
13:      for all minibatches  $b \subset \mathcal{B}$  of size  $|\mathcal{B}|/M$  do  $\triangleright$  gradient computation
14:        normalize the advantages  $A_t^a$  over  $b$  to zero mean and unit variance
15:         $\rho_t^i \leftarrow \pi_\theta(a_t^i | \tilde{o}_t^i) / \pi_{\theta_{old}}(a_t^i | \tilde{o}_t^i)$ 
16:         $\mathcal{L} \leftarrow -\langle \min(\rho Adv, \text{clip}(\rho, 1 \pm \epsilon) Adv) \rangle_m + c_v \langle \frac{1}{2} (V_\theta - \hat{V})^2 \rangle_m - c_e \langle \mathcal{H}[\pi_\theta] \rangle_m$ 
17:        update  $(\theta, \phi)$  by gradient descent on  $\mathcal{L}$  (Adam, gradient norm clipped)
18:      if  $\widehat{\text{KL}}(\pi_{\theta_{old}} \| \pi_\theta) > \kappa$  then
19:        break  $\triangleright$  early stopping on excessive policy change
```

Thus, the reward function used in this work is based on the distance of the agent from the closest point of the target area:

$$\phi(s) = 10 \cdot \left(1 - \frac{d}{diag}\right) \quad (4.4)$$

where d is the distance of the agent from the closest point of the target area and $diag$ is the diagonal of the domain, calculated as $\sqrt{L_x^2 + L_y^2 + L_z^2}$. Moreover, at success the agent receives a bonus of 20.0, which makes the final reward function:

$$r_t = \begin{cases} \gamma\phi(s') - \phi(s) & \text{if not success} \\ 20.0 & \text{if success} \end{cases} \quad (4.5)$$

4.5 Results

The trained policies were eight in total, given by the combination of the two algorithms (IPPO and MAPPO), with and without communication, and with two different numbers of agents (2 and 4). Training of the policies with 2 agents was done for 10M timesteps and took 3-5 hours each, Figure 4.1 shows the episodic return, length and success rate during training. Training was performed on an M3 chip with 10 cores and 16GB of RAM.

As can be seen, the agents start acting randomly, with a low success rate and a high episode length. As the training progresses, the agents learn to behave correctly and start

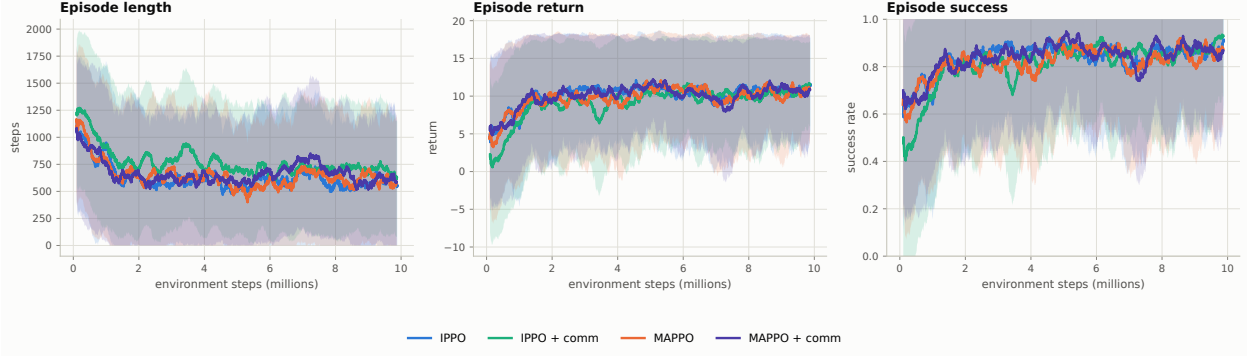


Figure 4.1: Training curves for 2 agent policies.

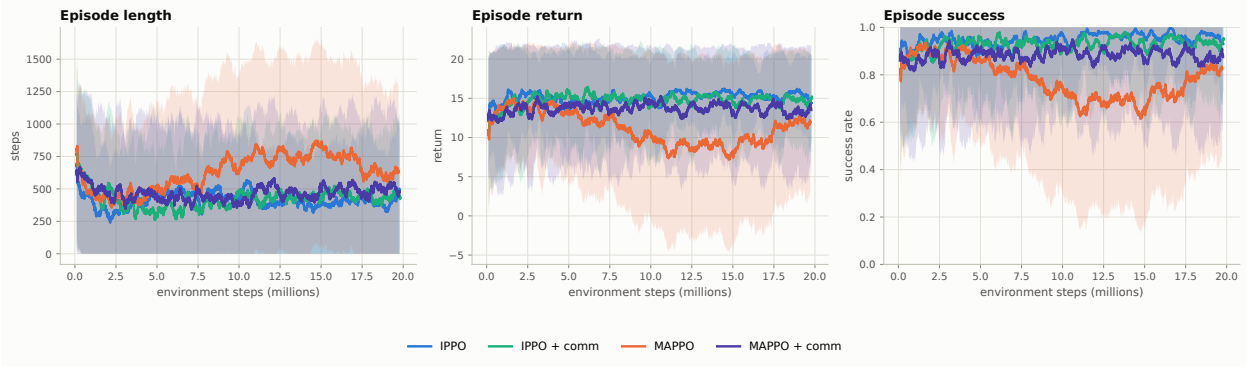


Figure 4.2: Training curves for 4 agent policies.

reaching higher returns and success rates, with lower episode lengths. One small caveat is that since the target zone is sampled randomly at each episode, it may be so large that the agents just need to learn to go in the target depth to reach the goal, creating a scenario where pure spawn luck can lead to success. This is reflected much more in the training curves of the 4 agent policies, where the number of agents is higher and so the chance of one of them spawning close to the target zone is much higher. In fact, Figure 4.2 shows almost flat curves, as if agents already start behaving optimally, even if in reality the agents start randomly but converge very fast due to this mechanism. This, unfortunately, is an intrinsic limitation of the task, since the target is an area and not a single point, and trying to avoid this by sampling a smaller region would introduce bias in the training and create a false optimal policy. Training of the policies with 4 agents was done for 20M timesteps, with minibatch size of 48 and took 6-8 hours each.

Figure 4.3 shows the trajectories developed during an episode by the IPPO policy without communication and using 2 agents. They are initialized with *max_dist*, and it is possible to see that one is actually trying to reach the target area the way one might expect, with a clean and healthy trajectory, while the other just went to the target depth and then stopped there, as if there is no incentive to move from there.

Figure 4.4, instead, shows the trajectories developed during an episode by the IPPO

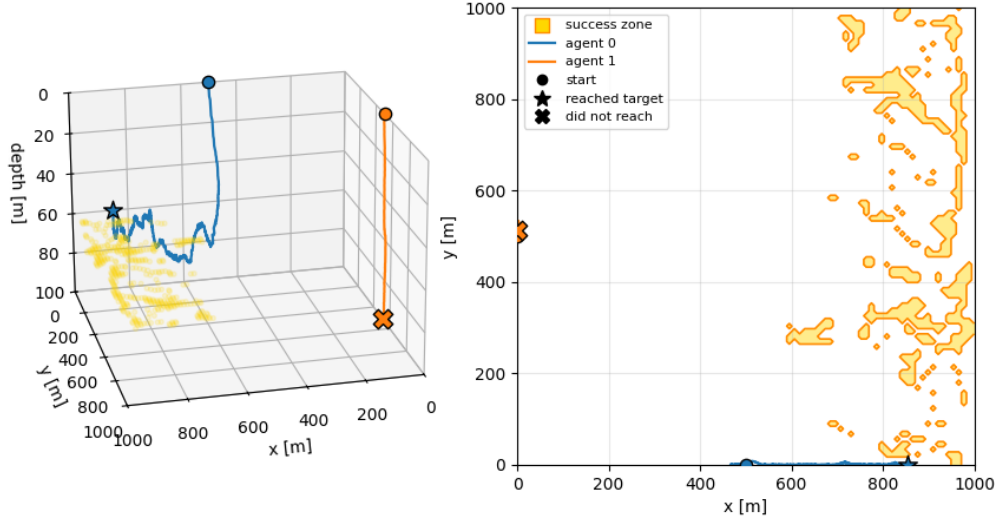


Figure 4.3: Trajectories of two agents initialized with max_dist and using an IPPO policy without communication.

policy without communication and using 4 agents. From this figure, it is clear that the agents are trying to reach the target area, probably by following the filaments in the salinity field, but that they don't know where the target area is.

The developed policies create non-intuitive behaviors, which are difficult to understand if the whole environment is not visible. In fact, being it complex and made of currents, agents have developed a strategy that seems like following the salinity filaments, which are zones in the environment that contain more or less the same salinity value, and the target area should be located in one of them.

The results on the test set are shown in Figure 4.5. As can be seen, the success rate is higher when the agents are randomly initialized in the environment, which is the same scenario they were trained on, and when they are initialized uniformly in the spawning area. Moreover, when the target area is small, i.e. when the sampled point is rejected until the salinity value is in the extreme 5% of its distribution at that depth layer, the success rate is lower, since it is a more difficult scenario. All the policies perform better than a random agent, even though in certain cases like spawning at the origin the success rate is comparable to the random agent.

Overall, IPPO without communication performs better with 4 agents, while MAPPO is comparable to it with 2 agents. This shows that the use of a centralized critic is not always beneficial, and that independent agents in this task are an optimal solution. The use of communication, instead, is beneficial in the harder scenarios and when agents are distributed in the environment. In the following chapter, the results are analyzed in more detail, and the effect of each component is analyzed in more depth.

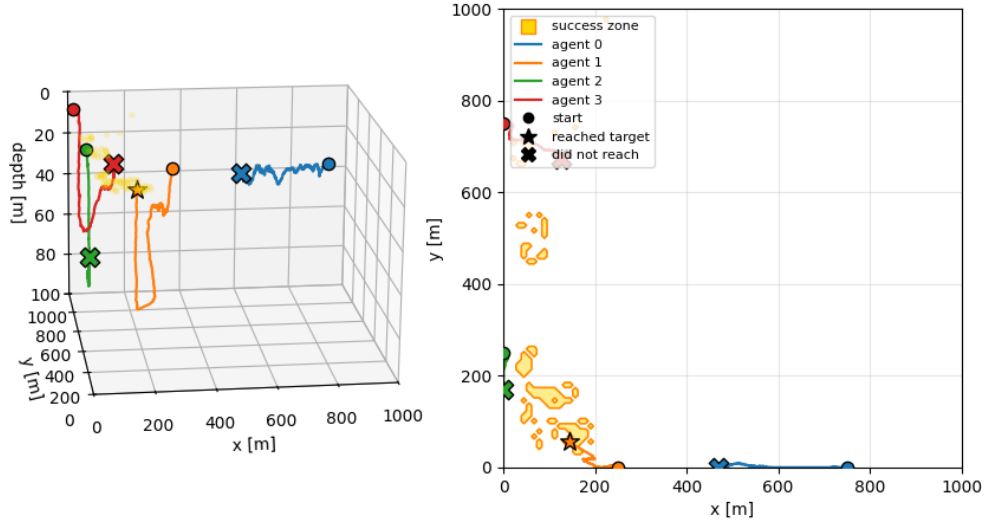


Figure 4.4: Trajectories of 4 agents initialized with *max_dist* and using a MAPPO policy without communication.

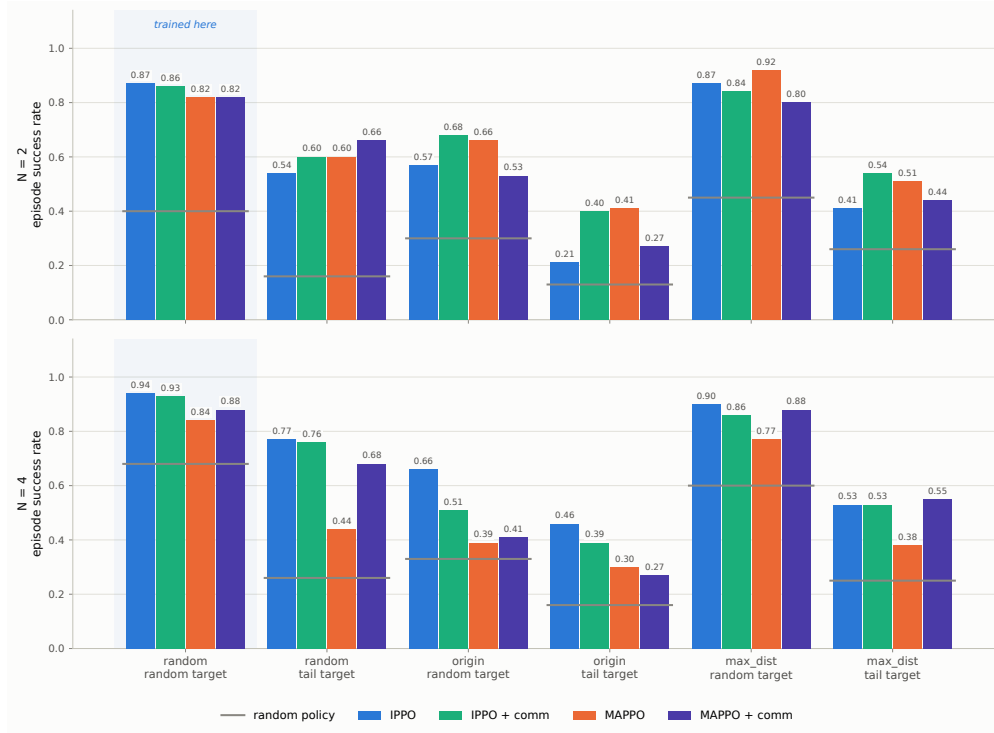


Figure 4.5: Success rate for the different test scenarios.

Chapter 5

Experiments

A set of experiments was designed to evaluate the performance and usefulness of the components of the proposed framework. For example, the effect of communication and CTDE on top of independent agents has not been yet evaluated, the increase of the number of agents seems not to bring any benefit except more coverage, and different scenarios have not been tested. Thus, in this chapter, the results and relative analysis of the experiments are presented.

Experiments are made of a set of 100 episodes each, and in each experiment some parameters are modified so that their effect can be evaluated. These parameters are:

- Communication radius: $\{\infty, 250, 0\}$ meters. This tests if communication is effectively used by the agents or if they perform better without it.
- Target area selection: $\{random, tail\}$. When selected randomly, the target values are taken from a randomly sampled point in the environment, which could lead to a target area that is quite large. When selected from the tail of the salinity distribution at that depth layer, the target area is definitely smaller, and the task is more difficult;
- Agent initialization: $\{random, origin, max_dist\}$. When initialized randomly, each agent spawns in a random point of the environment. When initialized at the origin, each agent spawns in the same point $(0, 0)$. When initialized with a maximum distance, the agents are spawned uniformly in the spawning area, which is defined by the south and west boundaries at surface level.

In the following sections, each experiment is analyzed carefully.

5.1 Communication and CTDE

Prior to any experiment, one can hypothesize that communication and CTDE, being them two ways of letting the agents share information, should help with difficult targets and have a more pronounced effect when the number of agents is increased. In Figure 5.1, it is possible to notice that communication sort of follows this idea, since it has a higher effect with a larger number of agents, and in general the effect is more pronounced with the harder targets. When targets are sampled randomly, there is not much evidence of communication effectiveness,

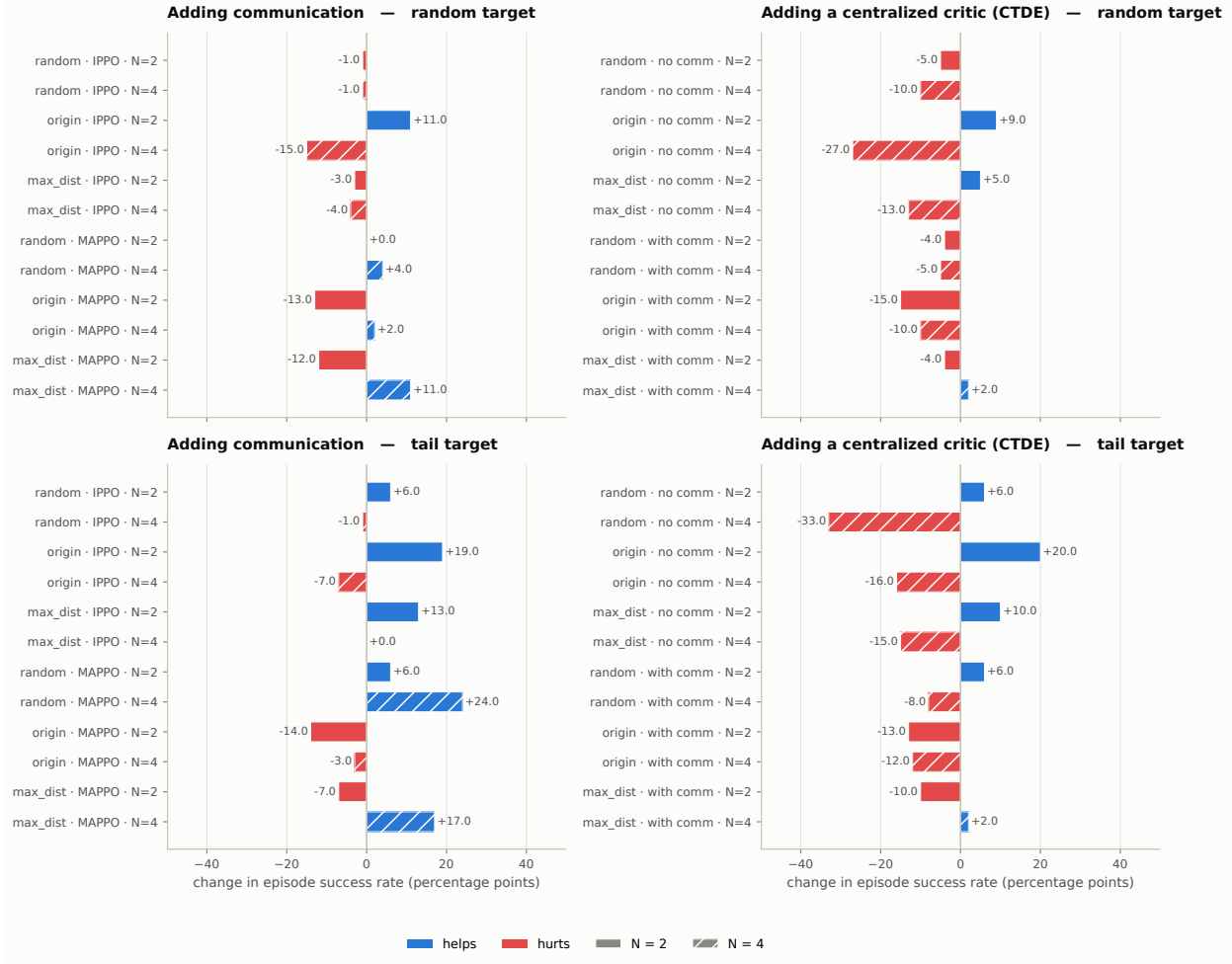


Figure 5.1: Results of using communication (left column) and CTDE (right column). The first row refers to scenarios with targets sampled randomly, while the second row refers to scenarios with targets sampled from the tail of the salinity distribution at sampled depth.

but when target areas are smaller, communication helps achieving success rates even 24% higher than the relative baseline. MAPPO seems to benefit from communication when more agents are present, while IPPO benefits from it when the target areas are small. Regarding CTDE, it seems that it hurts more then it helps. In fact, it seems to bring benefits only in the cases where agents don't have communication, signalling that the two mechanisms are complementary and should not be used together. Also, it seems that CTDE is more effective when the number of agents is small, and when they are initialized either in the same point of uniformly in the spawning area.

5.2 Agent initialization

How agents are initialized in the environment matters. Not only because they are able to cover more area but also because of the effect of communication. Figure 5.2 shows the success rate and the time to success for the different initialization methods that were tested. In general, the best results are obtained when agents are initialized randomly in the environment, since they were trained on that scenario to avoid introducing bias. The other two methods are more realistic, since they simulate the release of real AUVs from the land. The origin initialization, which spawns all the agents at the same point, is the worst case scenario in all cases. This is because the agents, made of the same policy, will tend to act the same way under the same circumstances, and if they start at the same point, they are in similar states from the beginning. That way, they cover less area and the effect of communication is less pronounced since they can not know the target values at distant points in space, but are limited to close points' values. The uniform initialization at the land boundaries is a better case, and it sort of reflects the random initialization, since results are similar. In this case, agents are naturally spread in the environment, and they can communicate to the others the target values they find, which helps the peers to know better about the environment even at distant points.

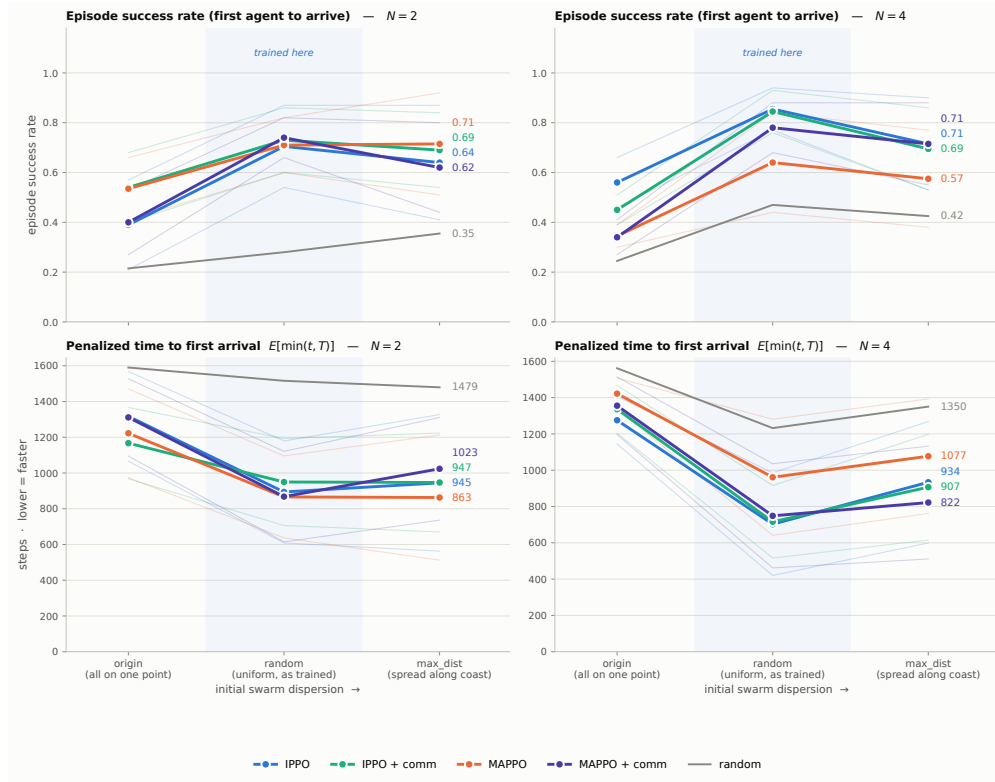


Figure 5.2: Effect of different initialization methods on success rate (first row) and steps to success (second row). The first column refers to 2 agent tests, while the right one to 4 agent tests. The shadowed lines are the marginalization of the bold ones on the two target methods, where the upper ones refer to random targets.

5.3 Difficult targets

Since in this task the target to search for is an area and not a point, and it has been shown that on average it is quite large, what happens if a smaller area is used as a target? To carry on this experiment, a random point is sampled in the environment until the salinity value is in the extreme 5% of its distribution at that depth layer. As Figure 5.3 shows, the success rate is naturally lower, but communication and CTDE help in these cases. In the figure, the plots show the fraction of episodes solved in function of the step budget, with 2 and 4 agents, in the random and tail targets. In the upper left plot, it is noticeable that the policies with communication or CTDE perform better than independent agents, which is not the case with random sampled targets. In the 4 agents case, this does not hold, since here independent agents are the optimal choice. As said before, this is probably a natural training limitation of this task, where the use of large zones as targets limit the learning of cooperation and optimal policies as the number of agents is increased, holding the environment fixed. Moreover, the effect of communication and CTDE is more pronounced with a higher step budget, i.e., when agents have more time to explore and understand the environment for the search task. This is easily noticeable in the case of difficult targets, since the line plotted does not have a linear trend as shown in the random targets' ones.

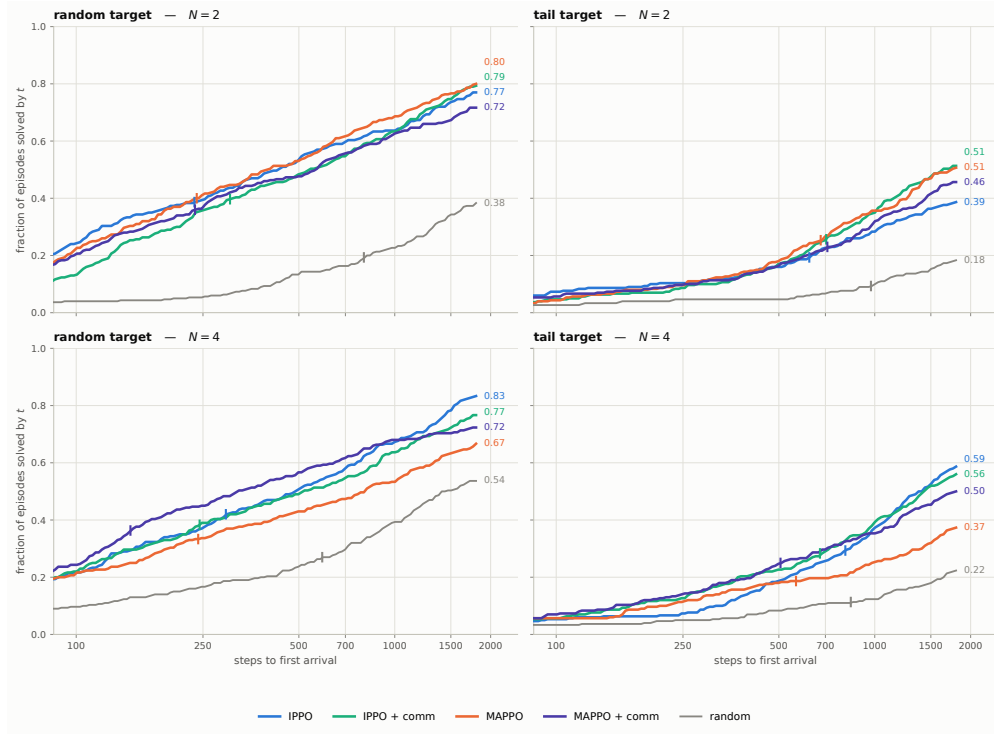


Figure 5.3: Results of using randomly sampled targets (left column) and targets sampled from the tail of the salinity distribution at sampled depth (right column). The first row refers to 2 agents' cases, while the second one to 4 agents' cases. The plots show the fraction of solved episodes as a function of the step budget.

5.4 Communication range

Agents were trained with an infinite value for the communication range either for simplicity and for the small environment domain, for which is assumed to exist a technology that let the agents communicate some bits for the targets' values and relative distance to each other. However, it is not said that communication is effectively used by the agents, and this experiment is made exactly for this reason.

As Figure 5.4 shows, with 2 agents the communication is effectively used, since the success rate drops as the communication range is cut. This indicates that the agents are able to use the advantage given by knowing the relative position of the other teammates and the target values they register to increase the success rate of a mission. However, using 4 agents it is the opposite, i.e., they are better without communication, and this has been extensively seen before, as independent agents in this case were the best option, and here is definitely confirmed. In fact, training a policy with 4 agents is limited by the nature of the problem, which therefore limits the learning of cooperation and communication, since the agents are trained on large target areas, which are easy to find.

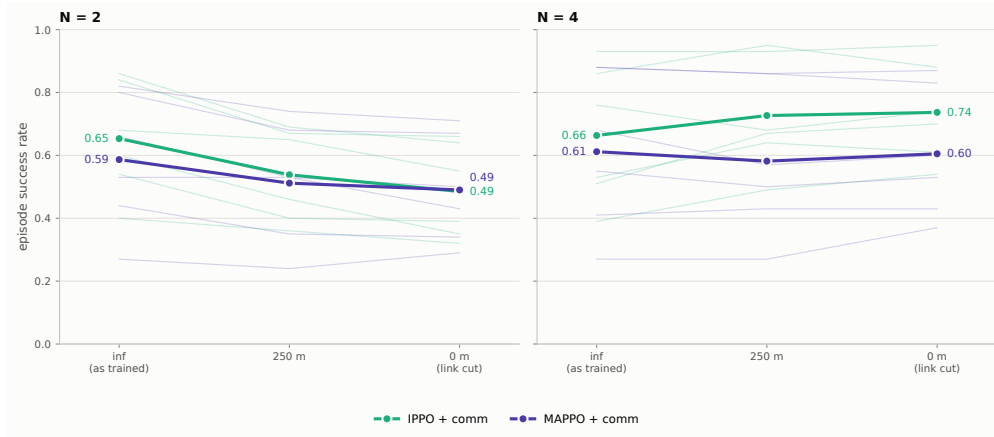


Figure 5.4: Effect of different communication ranges to the success rate on 2 agents (left plot) and 4 agents (right plot). The shadowed lines are the marginalization of the bold ones on the two target methods, where the upper ones refer to random targets.

5.5 Scaling N

When increasing the number of agents, the first thing that comes to mind is that the success rate should increase just because there are more agents to cover the area. However, it may not be the case for all the policies, due to the effect of communication and CTDE.

In Figure 5.5, it is possible to see the success rate and the penalized time to success as a function of the number of agents. The penalized time to success is computed as average time of the minimum between the time to success and the maximum number of steps at each episode, which is a metric that takes into account the number of episodes solved, since if an agent does not succeed in an episode, it is penalized with the maximum number of

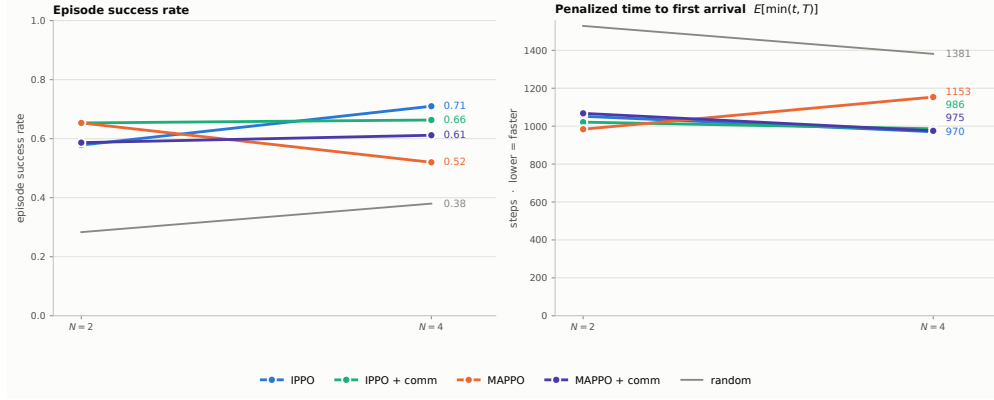


Figure 5.5: Average success rate using 2 agents (left plot) and 4 agents (right plot). The grey line refers to the random agent.

steps. In general, the success rate increases as the number of agents increases, and the time to success decreases as the number of agents increases, as one expects. The only exception is in this case MAPPO without communication, that seems to be worse with respect to all other policies. It has to be said, however, that this plot only considers the average value computed on all the scenarios, and is, therefore, a general metric. Previous figures, in fact, showed the effect of changing the number of agents in specific cases. For example, the use of communication on MAPPO was more effective on 4 agents than on 2 agents, as showed in Figure 5.1. At the end, however, all comes down to the limitation brought by the nature of the problem, i.e., the use of large target areas, which limited the learning of the policy with 4 agents and the subsequent use of communication and CTDE. More variables should be considered to solve this limitation.

5.6 Time to success

The success rate is not the only metric that can be used to evaluate the performance of a policy. In fact, it does not consider the time it takes for an agent to find the target zone. An agent that always succeeds but takes a long time to do so may be less useful than an agent that succeeds less often but finds the target quickly, since batteries don't have infinite capacity. Therefore, a thorough analysis on the times to reach the target is necessary. Figure 5.6 shows on the left charts the median time to reach target for the first agent only, while on the right charts the same metric but weighted for the number of episodes solved.

As can be noticed, the faster times can be found in the scenarios with random sampled targets and randomly initialized agents, the one the policies are trained on. The cases where targets are harder take longer to be solved, and the effect of communication in those cases is helpful, since it reduces the time to success. In the plot with 4 agents, there are larger times, but as can be seen in the right plot, weighted for the number of solved episodes the metric results in a value comparable to the one with 2 agents. In fact, the median time is higher because more difficult episodes are solved.

Scenarios where success is determined by all agents reaching the target have also been

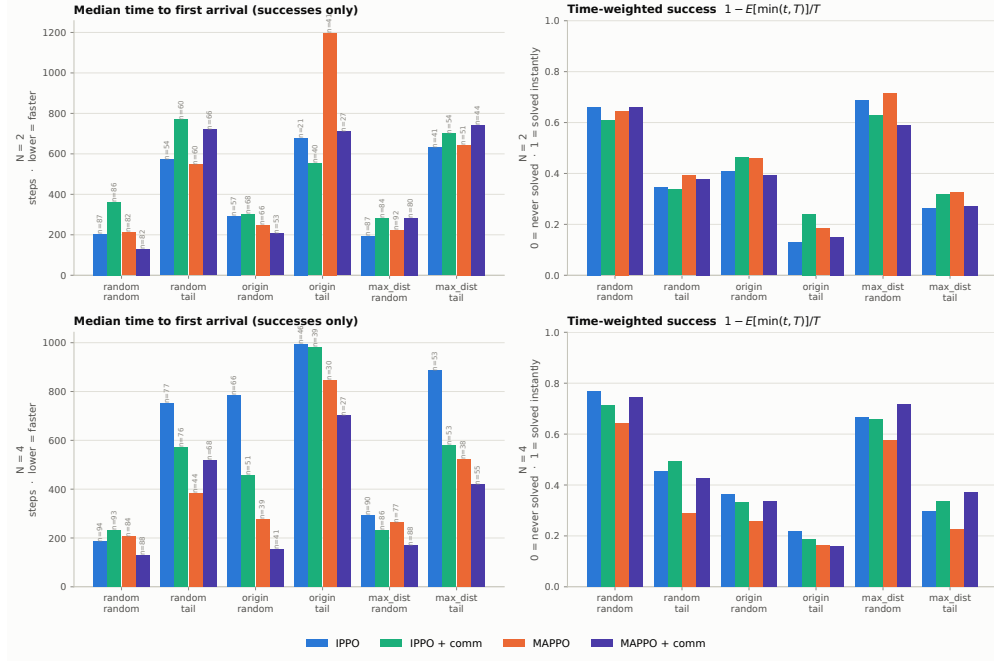


Figure 5.6: Median times to succeed for the first agent only (left column) and the relative normalized metric (right column), which is computed as $1 - \mathbb{E}(\min(t, T))/T$, where T refers to the *max_steps* parameter. The first row refers to 2 agents’ cases, while the second row to 4 agents’ ones.

considered. However, training on those specific scenarios is not trivial, since the reward must be carefully designed, since it is shared and some agents may not reach target. Therefore, the trained policies were evaluated on those cases, and the results are shown in Figures 5.7 for 2 agents’ policies and in Figure 5.8 for 4 agents’ policies. The figures show the probability of reaching target based on the rank of the agent, as a function of the step budget.

For 2 agents, the second agent reaches target with a lower probability than the first one, as expected. Moreover, the policies that results in better success rates are the ones that use either communication or CTDE. With an initialization of the agents in the same point, the metric is higher at lower budgets, but with uniform initialization the results are more robust between the policies.

Regarding the 4 agents’ policies, the results are more interesting. In fact, MAPPO with communication seems to perform better than all the other policies, in each case, and it is more pronounced in third and fourth agents with initialization at the same point. MAPPO alone here performs poorly, showing that communication helps later agents to understand that they need to go where the agents that already succeeded are.

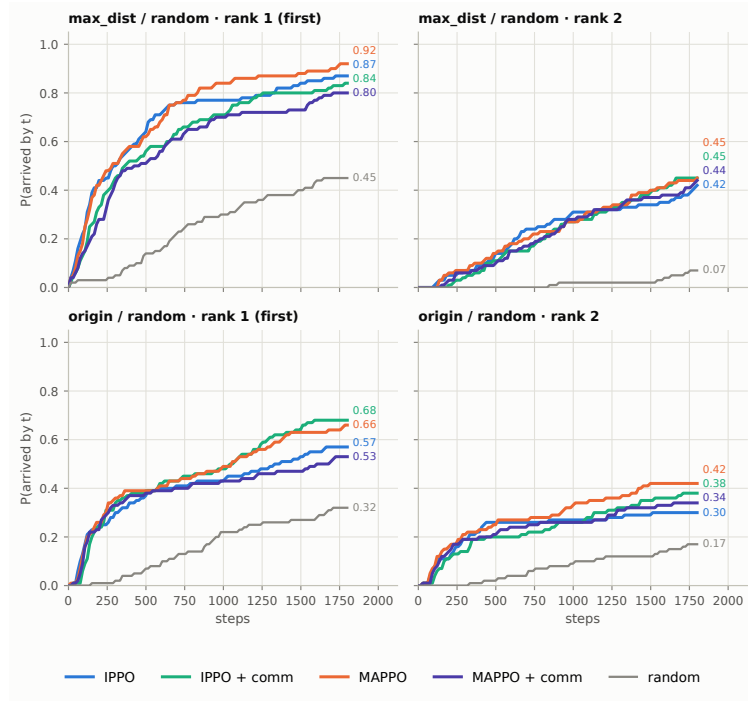


Figure 5.7: Probabilities to arrive at target by function of the number of steps for the first (left column) and second (right column) agent for the policies trained with 2 agents. The first row refers to cases where agents are initialized uniformly at land, the second one to cases where agents are initialized at the environment origin (0,0).

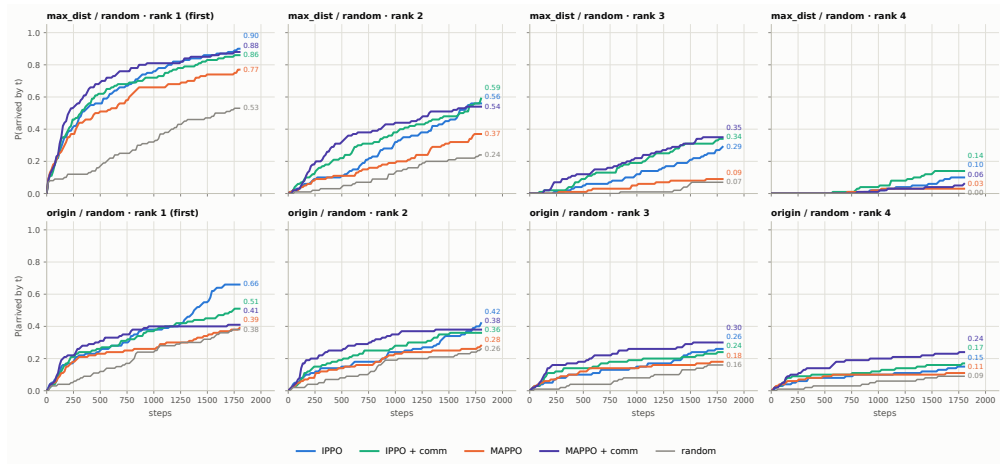


Figure 5.8: Probabilities to arrive at target by function of the number of steps for the first (first column), second (second column), third (third column) and fourth (fourth column) agent for the policies trained with 4 agents. The first row refers to cases where agents are initialized uniformly at land, the second one to cases where agents are initialized at the environment origin (0,0).

Chapter 6

Conclusions and Future Work

This work presented a comprehensive study on the application of Multi-Agent Reinforcement Learning (MARL) approaches to the problem of underwater search and navigation in an unknown, realistic and complex environment. The study focused on the application of multi-agent PPO variants, especially IPPO and MAPPO, with the addition of a communication mechanism to enhance the agents' performance in a partially observable environment. The results demonstrated the effectiveness of the proposed approaches in making the agents find the target area in the shortest amount of time. Different approaches lead to different results, even though independent agents are optimal choice if there is not prior information regarding the environment. In fact, the use of communication has been proved to help in difficult scenarios and in situations where agents are spread and thus find helpful having knowledge about other points in the environment, whether if they start close it does not help much. The centralized critic was not generally found helpful in this task, even though in some cases it helped achieving a better performance, especially with a lower number of agents and with a smaller target area. In this problem, since the target is defined as an area and usually it is quite large, training a policy with more than 2 agents was difficult and not optimal. Increasing to 4 agents, in fact, showed that independence leads to better performances overall, which is different from what one might expect. The effect of lucky spawning near the target zone, more probable in this case, has led to an optimal policy which favors the removal of communication, probably because using a larger state does not bring any advance with respect to the independent case, which is comprehensible. Efficiency is measured as the time to reach success, and in this task independent agents reached the worst performance overall, while MAPPO is the most efficient policy with 2 agents. This is explained by the fact that agents trained with a centralized policy are able to divide the work more efficiently than independent agents, which only care about success and reaching target. Coordinated agents, instead, trade some success for more efficiency, and that is proved by the time they take to reach the target zone. Finally, the way agents are initialized has been found to matter a lot, and the verdict is to use a uniform initialization to encode a natural spread in the environment. This could be informative for real deployment consideration, since it is not said that the cost of releasing the AUVs spread is lower than the what the difference in success rate brings, and that should be defined beforehand.

This problem is inherently difficult, since the target area depends on how the environment is made up and which variables are considered. In fact, in this case only two parameters

are considered for the target, but there could be more. Adding more constraints naturally complicates the learning problem, and there could be no optimal policy to navigate the environment in that case, since the more the variables that need to be taken into consideration, the more difficult would be for the agent to navigate using gradients and the higher will be the dimension of the state space. However, this should be the very next starting point for further analyze the effect of scaling the number of agents, since the dimension of the target area resulted to be the main problem that limited the training of an optimal policy with more than 2 agents. Moreover, other forms of communication and cooperation could be exploited and tested, for example with swarm capabilities that in this case were unhelpful due to the small environment and number of agents considered. If a larger number of agents are used, which complicates even further the learning problem, the consideration of techniques like PSO could help the agents avoid local optima generated from the more complex environment, since those techniques are more effective with large swarms.

To conclude, learning optimal policies in this task is definitely possible, even though the complexity of the real environment may be much higher than a simulated one like the one developed in this work. In fact, obstacles may be present and unexpected hazards are possible and should be taken into account. This work proved that MARL is a promising approach to solve problems in the underwater domain, like coral reef restoration and monitoring, which are vital for the health of the marine ecosystem. Further optimizations and improvements are needed to make the proposed approaches more robust and effective in real-world scenarios, but the results obtained in this work provide a solid foundation for future research in this area.

References

- [1] M. Omerspahic, H. Al-Jabri, S. A. Siddiqui, and I. Saadaoui. “Characteristics of Desalination Brine and Its Impacts on Marine Chemistry and Health, with Emphasis on the Persian/Arabian Gulf: A Review.” *Frontiers in Marine Science*, **9**, 2022, p. 845113. DOI: [10.3389/fmars.2022.845113](https://doi.org/10.3389/fmars.2022.845113).
- [2] E. E. Cordes, D. O. B. Jones, T. A. Schlacher, D. J. Amon, A. F. Bernardino, S. Brooke, R. Carney, D. M. DeLeo, K. M. Dunlop, E. G. Escobar-Briones, et al. “Environmental Impacts of the Deep-Water Oil and Gas Industry: A Review to Guide Management Strategies.” *Frontiers in Environmental Science*, **4**, 2016. Quantified impact footprints: cuttings 200–300 m from well-head, produced water 1–2 km., p. 58. DOI: [10.3389/fenvs.2016.00058](https://doi.org/10.3389/fenvs.2016.00058).
- [3] C. Sheppard, M. Al-Husiani, F. Al-Jamali, F. Al-Yamani, R. Baldwin, J. Bishop, F. Benzoni, E. Dutrieux, N. K. Dulvy, S. R. V. Durvasula, et al. “The Gulf: A Young Sea in Decline.” *Marine Pollution Bulletin*, **60**(1), 2010. Standard reference for the Gulf’s natural temperature/salinity extremes and its decline., pp. 13–38. DOI: [10.1016/j.marpolbul.2009.10.017](https://doi.org/10.1016/j.marpolbul.2009.10.017).
- [4] P. F. Sale et al. “The Growing Need for Sustainable Ecological Management of Marine Communities of the Persian Gulf.” *Ambio*, **40**(1), 2011. Umbrella reference: coastal construction is the single largest anthropogenic stressor; 60% of global seaborne oil transport; Gulf states produce 45% of desalinated water., pp. 4–17. DOI: [10.1007/s13280-010-0092-6](https://doi.org/10.1007/s13280-010-0092-6).
- [5] K. Zainal, I. Al-Madany, H. Al-Sayed, A. Khamis, S. Al Shuhaby, A. Al Hisaby, W. Elhoussiny, and E. Khalaf. “The Cumulative Impacts of Reclamation and Dredging on the Marine Ecology and Land-Use in the Kingdom of Bahrain.” *Marine Pollution Bulletin*, **64**(7), 2012, pp. 1452–1458.
- [6] J. A. Burt. “The Environmental Costs of Coastal Urbanization in the Arabian Gulf.” *City*, **18**(6), 2014, pp. 760–770. DOI: [10.1080/13604813.2014.962889](https://doi.org/10.1080/13604813.2014.962889).
- [7] F. Paparella, D. D’Agostino, and J. A. Burt. “Long-Term, Basin-Scale Salinity Impacts from Desalination in the Arabian/Persian Gulf.” *Scientific Reports*, **12**, 2022. Basin-scale salinity rise stays below 0.5 psu; the impact is regional and nearshore, not basin-wide., p. 20549. DOI: [10.1038/s41598-022-25167-5](https://doi.org/10.1038/s41598-022-25167-5).
- [8] J. Hadj-Hammou, A. Bartholomew, R. C. Bento, F. A. Mohamed, G. H. Cavalcante, and J. A. Burt. “Bleaching Impacts on the Last Remaining Acropora-Dominated Reefs in the United Arab Emirates.” *Diversity*, **17**(9), 2025, p. 610. DOI: [10.3390/d17090610](https://doi.org/10.3390/d17090610).

- [9] P. L. Erftemeijer, B. Riegl, B. W. Hoeksema, and P. A. Todd. “Environmental impacts of dredging and other sediment disturbances on corals: a review.” *Marine pollution bulletin*, **64**(9), 2012, pp. 1737–1765.
- [10] H. L. Burdett, R. Albright, G. L. Foster, T. Mass, T. M. Page, B. Rinkevich, V. Schoepf, J. Silverman, and N. A. Kamenos. “Including Environmental and Climatic Considerations for Sustainable Coral Reef Restoration.” *PLOS Biology*, **22**(3), 2024. 55% of restoration studies report no environmental parameters; calls for characterizing the environmental seascape before restoring., e3002542. DOI: [10.1371/journal.pbio.3002542](https://doi.org/10.1371/journal.pbio.3002542).
- [11] C. Mulà, C. J. A. Bradshaw, M. Cabeza, F. Manca, S. Montano, and G. Strona. “Restoration Cannot Be Scaled Up Globally to Save Reefs from Loss and Degradation.” *Nature Ecology & Evolution*, **9**(5), 2025. 30–40% of projects fail; site choice is driven by human proximity, not ecological suitability., pp. 822–832. DOI: [10.1038/s41559-025-02667-x](https://doi.org/10.1038/s41559-025-02667-x).
- [12] M. Talebi, S. Mahmud, A. Khalifa, and M. J. Islam. “BlueME: Robust Underwater Robot-to-Robot Communication Using Compact Magnetoelectric Antennas.” *IEEE Journal of Oceanic Engineering*, 2026. Early access. docs/BlueME.pdf. DOI: [10.1109/JOE.2026.3675822](https://doi.org/10.1109/JOE.2026.3675822).
- [13] M. Duarte, J. Gomes, V. Costa, T. Rodrigues, F. Silva, V. Lobo, M. M. Marques, S. M. Oliveira, and A. L. Christensen. “Application of Swarm Robotics Systems to Marine Environmental Monitoring.” In: *OCEANS 2016 – Shanghai*. Canonical “many cheap robots, decentralized control, local communication” argument. IEEE, 2016. DOI: [10.1109/OCEANSAP.2016.7485429](https://doi.org/10.1109/OCEANSAP.2016.7485429).
- [14] M. S. Couceiro, F. Mazloun, A. G. Araújo, R. P. Rocha, A. Bio, and G. Ferreira. “A New Paradigm for Marine Ecological Monitoring through Swarm Intelligence, Digital Twins, and Human–Swarm Interaction.” *Frontiers in Robotics and AI*, **13**, 2026, p. 1809951. DOI: [10.3389/frobt.2026.1809951](https://doi.org/10.3389/frobt.2026.1809951).
- [15] F. Berlinger, M. Gauci, and R. Nagpal. “Implicit Coordination for 3D Underwater Collective Behaviors in a Fish-Inspired Robot Swarm.” *Science Robotics*, **6**(50), 2021, eabd8668. DOI: [10.1126/scirobotics.abd8668](https://doi.org/10.1126/scirobotics.abd8668). URL: <https://www.science.org/doi/10.1126/scirobotics.abd8668>.
- [16] M. O. Berild, Y. Ge, J. Eidsvik, G.-A. Fuglstad, and I. Ellingsen. “Efficient 3D Real-Time Adaptive AUV Sampling of a River Plume Front.” *Frontiers in Marine Science*, **10**, 2024. Adaptive AUV sampling of a salinity front; preplanning is impossible when the feature location is unknown., p. 1319719. DOI: [10.3389/fmars.2023.1319719](https://doi.org/10.3389/fmars.2023.1319719).
- [17] P. Stankiewicz, Y. T. Tan, and M. Kobilarov. “Adaptive Sampling with an Autonomous Underwater Vehicle in Static Marine Environments.” *Journal of Field Robotics*, **38**(4), 2021, pp. 572–597. DOI: [10.1002/rob.22005](https://doi.org/10.1002/rob.22005).
- [18] J. A. Farrell, S. Pang, and W. Li. “Chemical Plume Tracing via an Autonomous Underwater Vehicle.” *IEEE Journal of Oceanic Engineering*, **30**(2), 2005, pp. 428–442. DOI: [10.1109/JOE.2005.850284](https://doi.org/10.1109/JOE.2005.850284).

- [19] E. Insaghi. “Adaptive Control of Underwater Artificial Swarm for Monitoring Marine Ecosystems.” Supervisors: A. Celani, G. De Masi. docs/Edoardo_Thesis.pdf. Master’s thesis. University of Trieste, Department of Mathematics, Informatics and Geosciences, 2025.
- [20] S. V. Albrecht, F. Christianos, and L. Schäfer. *Multi-agent reinforcement learning: Foundations and modern approaches*. MIT Press, 2024.
- [21] J. Schulman, S. Levine, P. Moritz, M. I. Jordan, and P. Abbeel. “Trust Region Policy Optimization.” In: *Proceedings of the 32nd International Conference on Machine Learning (ICML)*. Vol. 37. Proceedings of Machine Learning Research. docs/TRPO.pdf. PMLR, 2015, pp. 1889–1897. arXiv: [1502.05477](https://arxiv.org/abs/1502.05477).
- [22] K. Zhang, Z. Yang, and T. Başar. “Multi-agent reinforcement learning: A selective overview of theories and algorithms.” *Handbook of reinforcement learning and control*, 2021, pp. 321–384.
- [23] *SWIM: Underwater SWarm sIMulator*. URL: <https://github.com/aghilmort/swim> (visited on 08/02/2026).
- [24] J.-M. Lee, C.-J. Lee, Y.-S. Kim, G.-G. Choi, and J.-M. Lew. “Determination of global ice loads on the ship using the measured full-scale motion data.” *International Journal of Naval Architecture and Ocean Engineering*, **8**(4), 2016, pp. 301–311. ISSN: 2092-6782. DOI: <https://doi.org/10.1016/j.ijnaoe.2016.03.006>. URL: <https://www.sciencedirect.com/science/article/pii/S2092678216000431>.
- [25] G. L. Wagner et al. “High-level, high-resolution ocean modeling at all scales with Oceananigans.” *arXiv preprint*, 2025. DOI: [10.48550/arXiv.2502.14148](https://doi.org/10.48550/arXiv.2502.14148). arXiv: [2502.14148](https://arxiv.org/abs/2502.14148).
- [26] P. A. Alarcon Granadeno, T. Chambers, and J. Cleland-Huang. *Multi-Source Plume Tracing via Multi-Agent Reinforcement Learning*. docs/2505.08825v1.pdf. 2025. DOI: [10.48550/arXiv.2505.08825](https://doi.org/10.48550/arXiv.2505.08825). arXiv: [2505.08825](https://arxiv.org/abs/2505.08825) [cs.MA].
- [27] L. Tan, H. Zhang, Y. Liu, T. Yuan, X. Jiang, and Z. Shang. “An Adaptive Q-Learning Based Particle Swarm Optimization for Multi-UAV Path Planning.” *Soft Computing*, **28**, 2024. docs/An adaptoive Q learning based Partcile Swarm Optimization.pdf, pp. 7931–7946. DOI: [10.1007/s00500-024-09691-2](https://doi.org/10.1007/s00500-024-09691-2).
- [28] Z. Ma and Z. Liu. “An Improved Q-Learning Algorithm with Particle Swarm Optimization for Path Planning.” In: *2024 6th International Conference on Frontier Technologies of Information and Computer (ICFTIC)*. docs/An_Improved_Q-Learning_Algorithm_with_Particle_Swarm_Optimization_for_Path_Planning.pdf. IEEE, 2024, pp. 1662–1667. DOI: [10.1109/ICFTIC64248.2024.10913234](https://doi.org/10.1109/ICFTIC64248.2024.10913234).
- [29] R. A. Saeed, E. S. Ali, M. Abdelhaq, R. Alsaqour, F. R. A. Ahmed, and A. M. E. Saad. “Energy Efficient Path Planning Scheme for Unmanned Aerial Vehicle Using Hybrid Generic Algorithm-Based Q-Learning Optimization.” *IEEE Access*, **12**, 2024. docs/Energy_Efficient_Path_Planning_Scheme_for_Unmanned_Aerial_Vehicle_Using_Hybrid_Generic_Algorithm-Based_Q-Learning_Optimization.pdf, pp. 13400–13417. DOI: [10.1109/ACCESS.2023.3344455](https://doi.org/10.1109/ACCESS.2023.3344455).

- [30] S. I. A. Meerza, M. Islam, and M. M. Uzzal. “Q-Learning Based Particle Swarm Optimization Algorithm for Optimal Path Planning of Swarm of Mobile Robots.” In: *2019 1st International Conference on Advances in Science, Engineering and Robotics Technology (ICASERT)*. docs/Q-Learning_Based_Particle_Swarm_Optimization_Algorithm_for_Optimal_Path_Planning_of_Swarm_of_Mobile_Robots.pdf. IEEE, 2019, pp. 1–5. DOI: [10.1109/ICASERT.2019.8934450](https://doi.org/10.1109/ICASERT.2019.8934450).
- [31] D. Li, W. Yin, W. E. Wong, M. Jian, and M. Chau. “Quality-Oriented Hybrid Path Planning Based on A* and Q-Learning for Unmanned Aerial Vehicle.” *IEEE Access*, **10**, 2022. docs/Quality-Oriented_Hybrid_Path_Planning_Based_on_A_and_Q-Learning_for_Unmanned_Aerial_Vehicle.pdf, pp. 7664–7674. DOI: [10.1109/ACCESS.2021.3139534](https://doi.org/10.1109/ACCESS.2021.3139534).
- [32] W. Li, Y. Xiong, and Q. Xiong. “Reinforcement Learning-Guided Particle Swarm Optimization for Multi-Objective Unmanned Aerial Vehicle Path Planning.” *Symmetry*, **17**(8), 2025. docs/Reinforcement_Learning-Guided_Particle_Swarm_Optimization.pdf, p. 1292. DOI: [10.3390/sym17081292](https://doi.org/10.3390/sym17081292).
- [33] Y. Li, M. Ma, J. Cao, G. Luo, D. Wang, and W. Chen. “A Method for Multi-AUV Cooperative Area Search in Unknown Environment Based on Reinforcement Learning.” *Journal of Marine Science and Engineering*, **12**(7), 2024. docs/A_Method_for_Multi-AUV_Cooperative_Area_Search_in_Unknown_Environment_Based_on_Reinforcement_Learning.pdf, p. 1194. DOI: [10.3390/jmse12071194](https://doi.org/10.3390/jmse12071194).
- [34] L. Li, R. An, Z. Guo, and J. Gao. “Multi-AUV Cooperative Search for Moving Targets Based on Multi-Agent Reinforcement Learning.” *Journal of Marine Science and Engineering*, **13**(11), 2025. docs/Multi-AUV_Cooperative_Target_Search_Based_on_Reinforcement_Learning_and_Bio-Inspired_Algorithms..pdf, p. 2072. DOI: [10.3390/jmse13112072](https://doi.org/10.3390/jmse13112072).
- [35] Y. Hou, J. Zhao, R. Zhang, X. Cheng, and L. Yang. “UAV Swarm Cooperative Target Search: A Multi-Agent Reinforcement Learning Approach.” *IEEE Transactions on Intelligent Vehicles*, **9**(1), 2024. docs/UAV_Swarm_Cooperative_Target_Search_A_Multi-Agent_Reinforcement_Learning_Approach.pdf, pp. 568–578. DOI: [10.1109/TIV.2023.3316196](https://doi.org/10.1109/TIV.2023.3316196).
- [36] C. Schroeder de Witt, T. Gupta, D. Makoviichuk, V. Makoviyuchuk, P. H. S. Torr, M. Sun, and S. Whiteson. *Is Independent Learning All You Need in the StarCraft Multi-Agent Challenge?* IPPO. docs/IPPO.pdf (= docs/independent-learning.pdf). 2020. DOI: [10.48550/arXiv.2011.09533](https://doi.org/10.48550/arXiv.2011.09533). arXiv: [2011.09533](https://arxiv.org/abs/2011.09533) [cs.AI].
- [37] C. Yu, A. Velu, E. Vinitzky, J. Gao, Y. Wang, A. Bayen, and Y. Wu. “The Surprising Effectiveness of PPO in Cooperative Multi-Agent Games.” In: *Advances in Neural Information Processing Systems 35 (NeurIPS), Datasets and Benchmarks Track*. MAPPO. docs/MAPPO.pdf. 2022. arXiv: [2103.01955](https://arxiv.org/abs/2103.01955) [cs.LG].
- [38] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. *Proximal Policy Optimization Algorithms*. docs/ppo.pdf. 2017. DOI: [10.48550/arXiv.1707.06347](https://doi.org/10.48550/arXiv.1707.06347). arXiv: [1707.06347](https://arxiv.org/abs/1707.06347) [cs.LG].
- [39] A. Y. Ng, D. Harada, and S. Russell. “Policy invariance under reward transformations: Theory and application to reward shaping.” In: *ICML*. Vol. 99. Citeseer. 1999, pp. 278–287.